

ĐẠI HỌC BÁCH KHOA HÀ NỘI
KHOA TOÁN - TIN



BÁO CÁO CUỐI KÌ
KIẾN TRÚC MÁY TÍNH

Thiết kế bộ giải mã ảnh đa phương tiện JPEG

Giảng viên hướng dẫn: PGS. TS Nguyễn Đình Hân
Nhóm sinh viên thực hiện: Nhóm 10

<i>Họ và tên</i>	<i>MSSV</i>
Cao Hải An	20237287
Phạm Thành Đạt	20237308
Nguyễn Quang Dương	20237320
Bùi Duy Hiếu	20237329
Bùi Hằng Nga	20237371

Hà Nội, Ngày 6 tháng 9 năm 2026

Lời mở đầu

Học phần Kiến trúc máy tính là một trong những trụ cột quan trọng nhất của ngành kỹ thuật máy tính, giúp sinh viên không chỉ hiểu về cách thức vận hành của phần cứng mà còn hình thành tư duy thiết kế hệ thống tối ưu. Với mong muốn vận dụng những kiến thức lý thuyết nền tảng ấy vào một bài toán thực tế đầy thách thức, nhóm chúng em đã quyết định lựa chọn thực hiện đề tài “Thiết kế bộ giải mã ảnh đa phương tiện JPEG”.

Thông qua đề tài này, chúng em không chỉ mong muốn tạo ra một sản phẩm hoạt động được mà còn coi đây là cơ hội quý báu để rèn luyện kỹ năng phân tích luồng dữ liệu, tư duy tổ chức kiến trúc phần cứng và kỹ năng làm việc nhóm. Quá trình nghiên cứu và hiện thực hóa chuẩn nén ảnh JPEG đã giúp chúng em hiểu rõ hơn sự gắn kết chặt chẽ giữa giải thuật phần mềm và kiến trúc phần cứng bên dưới.

Trong quá trình thực hiện, mặc dù đã dành nhiều tâm huyết và nỗ lực tìm tòi, nhưng do khối lượng kiến thức của môn học rất rộng và thời gian có hạn, bản báo cáo này khó tránh khỏi những thiếu sót. Nhóm chúng em rất mong nhận được những lời nhận xét, góp ý quý báu từ Thầy để có thể khắc phục các hạn chế và hoàn thiện tư duy chuyên môn tốt hơn trong tương lai.

Lời cuối cùng, chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến thầy PGS. TS. Nguyễn Đình Hân. Cảm ơn Thầy đã tận tình giảng dạy, truyền cảm hứng và định hướng tư duy cho chúng em trong suốt quá trình học tập môn Kiến trúc máy tính cũng như trong thời gian thực hiện đề tài này.

Từ nhóm sinh viên thực hiện.

Mục lục

1	Giới thiệu	4
1.1	Giới thiệu đề tài	4
1.2	Mục tiêu của đề tài	5
1.3	Ứng dụng thực tế của đề tài	5
1.4	Phạm vi và giới hạn hệ thống	6
1.4.1	Phạm vi hệ thống	6
1.4.2	Giới hạn hệ thống	7
1.5	Công cụ sử dụng:	7
1.5.1	Ngôn ngữ mô tả phần cứng Verilog	7
1.5.2	Công cụ mô phỏng Icarus Verilog / ModelSim	8
1.5.3	Ngôn ngữ Python cho kiểm chứng kết quả	8
2	Tổng quan về chuẩn JPEG	9
2.1	Giới thiệu chuẩn JPEG	9
2.2	Nguyên lý nén ảnh JPEG	9
2.2.1	Chuyển đổi không gian màu	9
2.2.2	Biến đổi Cosin rời rạc	11
2.2.3	Lượng tử hóa	11
2.2.4	Sắp xếp Zig-zag	13
2.2.5	Mã hóa Huffman	14
2.3	Nguyên lý giải mã JPEG	15
2.3.1	Định nghĩa và Cấu trúc chung của Marker	15
2.3.2	Một số Marker quan trọng	17
3	Kiến trúc tổng thể bộ giải mã JPEG	18
3.1	Cấu trúc phân cấp và Tổ chức các Module	18
3.2	Thiết kế Module Top-level và Logic điều phối	19
3.2.1	Quản lý "Biến chung" và Kết nối liên tầng	19
3.2.2	Logic điều phối Pipeline và Đồng bộ hóa nội bộ	20
3.2.3	Bảng tóm tắt vai trò và Giao tiếp giữa các file	20
4	Thiết kế chi tiết từng khối chức năng	21
4.1	Byte Stream Parser & Marker Decoder	21
4.1.1	Máy trạng thái quản lý Marker (FSM Control)	21
4.1.2	Xử lý các Segment quan trọng	21
4.2	Bitstream Reader & Byte Stuffing	22
4.2.1	Cơ chế loại bỏ Byte Stuffing (0xFF00)	22
4.2.2	Quản lý Thanh ghi dịch và Giao tiếp	23
4.3	Bộ giải mã Entropy	24
4.3.1	Cấu trúc bảng Huffman	24
4.3.2	Thuật toán so khớp bit	24
4.3.3	Xử lý hệ số DC / AC	25
4.3.4	So sánh với thuật toán phần mềm	25
4.4	Giải mã hệ số & Dequantization	25
4.4.1	Bảng lượng tử	26
4.4.2	Phép nhân ngược lượng tử	26
4.5	Xấp xếp Zig-Zag	26

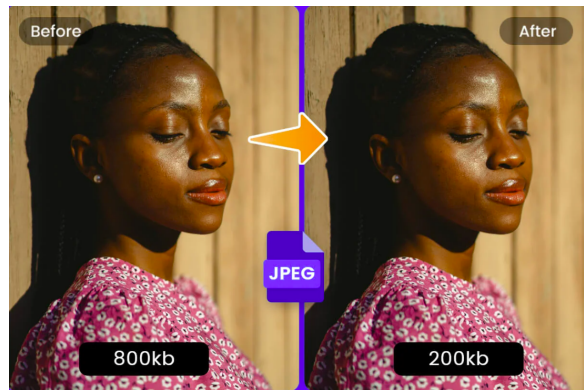
4.5.1	Mapping chỉ số	27
4.5.2	Cài đặt phần cứng	27
4.6	IDCT (Inverse Discrete Cosine Transform)	27
4.6.1	Cơ sở lý thuyết	27
4.6.2	Kiến trúc phần cứng	28
4.7	Độ chính xác số nguyên và Định dạng Fixed-point	28
4.7.1	Định dạng Q8 và Hệ số tỉ lệ	28
4.7.2	Phân tích sai số lượng tử hóa	29
4.7.3	Phân tích độ rộng đường dữ liệu	29
4.8	Chuyển đổi màu (YCbCr $\rightarrow$ RGB)	30
4.8.1	Mục đích của module chuyển đổi YCbCr sang RGB	30
4.8.2	Xử lý dữ liệu đầu vào từ khối IDCT	30
4.8.3	Thuật toán chuyển đổi không gian màu YCbCr sang RGB	31
5	Thiết kế môi trường kiểm thử	33
5.1	Chiến lược kiểm thử (Testing Strategy)	33
5.2	Kịch bản mô phỏng	33
5.2.1	Đầu vào kiểm thử	33
5.2.2	Đầu ra mong muốn	34
5.3	Kiến trúc Testbench tổng thể	34
5.3.1	Tạo xung nhịp và tín hiệu reset	34
5.3.2	Cơ chế đọc dữ liệu ảnh đầu vào	35
5.3.3	Theo dõi quá trình phân tích Header	35
5.3.4	Thu thập và ghi dữ liệu pixel đầu ra	35
5.3.5	Cơ chế giám sát và kết thúc mô phỏng	35
6	Kết quả thực nghiệm và Đánh giá hệ thống	36
6.1	Kiểm chứng dữ liệu chi tiết từng khối (Log Files)	36
6.1.1	Khối phân tích Header	36
6.1.2	Khối giải mã Entropy	37
6.1.3	Khối quét Zigzag	37
6.1.4	Khối giải lượng tử	38
6.1.5	Khối biến đổi ngược	38
6.1.6	Khối tái tạo MCU	39
6.2	Phân tích gián đồ xung và Giao thức điều khiển	40
6.2.1	Giao thức Handshake tại Bitstream Reader	40
6.2.2	Phân tích quá trình chuyển đổi trạng thái (System Handover) .	41
6.2.3	Phân tích độ trễ Pipeline và Chất lượng tín hiệu đầu ra	42
6.3	Kết quả hiển thị cuối cùng	43
6.4	So sánh kết quả với phần mềm (Python / libjpeg)	43
6.4.1	Kết quả so sánh chi tiết	44
6.4.2	Đánh giá định lượng (MSE & PSNR)	45
6.5	Một số hạn chế và sai lệch của hệ thống	45
7	Kết luận và hướng phát triển	47
7.1	Kết quả đạt được	47
7.2	Đánh giá ưu điểm và hạn chế	47
7.3	Hướng phát triển trong tương lai	47

1 Giới thiệu

1.1 Giới thiệu đề tài

Trong thời đại công nghệ số phát triển mạnh mẽ, dữ liệu đa phương tiện ngày càng đóng vai trò quan trọng trong các hệ thống máy tính hiện đại. Ảnh số được sử dụng rộng rãi trong nhiều lĩnh vực như truyền thông, lưu trữ dữ liệu, giám sát an ninh, y tế, thị giác máy tính và các hệ thống nhúng. Tuy nhiên, ảnh số thường có dung lượng lớn, dẫn đến những yêu cầu khắt khe về tài nguyên lưu trữ, băng thông truyền tải và năng lực xử lý của hệ thống.

Để giải quyết bài toán này, các kỹ thuật nén ảnh đã được nghiên cứu và áp dụng nhằm giảm kích thước dữ liệu trong khi vẫn duy trì chất lượng hình ảnh ở mức chấp nhận được. Trong số các chuẩn nén ảnh hiện nay, JPEG (Joint Photographic Experts Group) là chuẩn nén ảnh tĩnh phổ biến nhất, được sử dụng rộng rãi trong nhiều thiết bị và hệ thống khác nhau, từ máy ảnh số, điện thoại di động cho đến các ứng dụng web và hệ thống nhúng.



Hình 1: Ảnh minh họa ảnh nén JPEG.

Chuẩn JPEG dựa trên việc khai thác đặc điểm cảm nhận thị giác của con người kết hợp với các phương pháp xử lý tín hiệu số như biến đổi cosin rời rạc (DCT), lượng tử hóa và mã hóa entropy. Quá trình giải mã ảnh JPEG bao gồm nhiều bước xử lý liên tiếp như phân tích header, giải mã Huffman, khôi phục các hệ số DCT, giải lượng tử, biến đổi IDCT, sắp xếp hệ số theo thứ tự zigzag và chuyển đổi không gian màu từ YCbCr sang RGB. Đây là một chuỗi xử lý phức tạp, đòi hỏi sự phối hợp chặt chẽ giữa các khối chức năng trong hệ thống.

Dưới góc nhìn của học phần Kiến trúc máy tính, bộ giải mã ảnh JPEG là một ví dụ tiêu biểu cho việc hiện thực hóa một thuật toán xử lý ảnh mức cao thành các khối xử lý phần cứng cụ thể. Hệ thống giải mã JPEG có kiến trúc dạng luồng dữ liệu (dataflow), bao gồm nhiều module hoạt động song song và tuần tự, phù hợp để phân tích các khía cạnh quan trọng như tổ chức datapath, control path, cơ chế đồng bộ, điều khiển trạng thái và tối ưu hiệu năng.

Việc nghiên cứu, thiết kế và mô phỏng bộ giải mã ảnh nén JPEG không chỉ giúp làm rõ mối liên hệ giữa thuật toán và kiến trúc phần cứng mà còn tạo điều kiện cho sinh viên tiếp cận quy trình thiết kế một hệ thống xử lý số hoàn chỉnh. Thông qua đề tài này, sinh viên có thể củng cố kiến thức về kiến trúc máy tính, đồng thời nâng cao tư duy hệ thống và kỹ năng thiết kế phần cứng mô phỏng.

1.2 Mục tiêu của đề tài

Mục tiêu tổng quát của đề tài: Nghiên cứu, phân tích và thiết kế mô phỏng một bộ giải mã ảnh nén JPEG dưới góc nhìn của kiến trúc máy tính, nhằm làm rõ cách tổ chức và hoạt động của các khối xử lý trong hệ thống.

Mục tiêu cụ thể bao gồm:

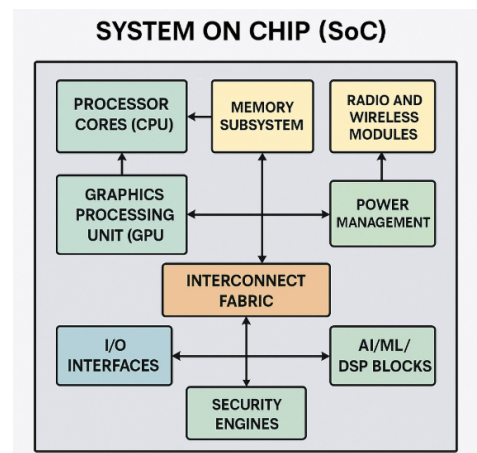
- Nghiên cứu nguyên lý hoạt động của chuẩn nén ảnh JPEG, đặc biệt là các bước trong quá trình giải mã ảnh.
- Phân tích cấu trúc và kiến trúc tổng thể của bộ giải mã JPEG, xác định chức năng và vai trò của từng khối xử lý trong hệ thống.
- Thiết kế và mô phỏng các module chính của bộ giải mã JPEG bằng ngôn ngữ mô tả phần cứng (Verilog), đảm bảo đúng luồng dữ liệu và tín hiệu điều khiển theo chuẩn.
- Tích hợp các module thành một hệ thống giải mã hoàn chỉnh và tiến hành mô phỏng để kiểm tra tính đúng đắn của thiết kế.
- Đánh giá kết quả mô phỏng, phân tích những ưu điểm và hạn chế của kiến trúc đề xuất, từ đó đề xuất các hướng cải tiến và mở rộng trong tương lai.
- Vận dụng kiến thức của học phần Kiến trúc máy tính vào một bài toán thực tế, qua đó nâng cao kỹ năng phân tích, thiết kế và trình bày báo cáo kỹ thuật.

1.3 Ứng dụng thực tế của đề tài

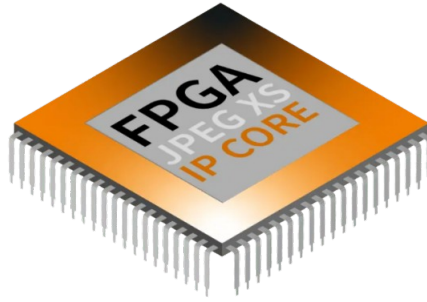
Trong các hệ thống điện tử hiện đại, bộ giải mã ảnh nén JPEG thường được tích hợp trực tiếp vào phần cứng dưới dạng các khối xử lý chuyên dụng (hardware accelerator) nhằm đáp ứng yêu cầu về hiệu năng, độ trễ và tiết kiệm năng lượng. Việc hiện thực hóa bộ giải mã JPEG ở mức phần cứng giúp giảm tải cho bộ xử lý trung tâm (CPU), đồng thời cho phép xử lý ảnh theo thời gian thực trong nhiều ứng dụng khác nhau.

Trong các hệ thống nhúng và thiết bị tiêu dùng như máy ảnh số, máy in, máy quét (scanner) và thiết bị đa phương tiện, bộ giải mã JPEG được tích hợp trong các SoC (System-on-Chip) để giải mã ảnh trực tiếp từ bộ nhớ hoặc thiết bị lưu trữ. Dữ liệu ảnh sau khi giải mã được ghi vào bộ đệm khung hình (frame buffer) và chuyển tới các khối hiển thị hoặc xử lý tiếp theo.

Đối với các thiết bị di động như điện thoại thông minh và máy tính bảng, bộ giải mã JPEG thường tồn tại dưới dạng IP core trong chip xử lý ứng dụng (Application Processor). Nhờ sử dụng phần cứng chuyên dụng, quá trình giải mã ảnh có thể được thực hiện với mức tiêu thụ năng lượng thấp hơn đáng kể so với việc thực hiện hoàn toàn bằng phần mềm, từ đó góp phần kéo dài thời lượng pin



Hình 2: Kiến trúc SoC bộ giải mã JPEG được tích hợp xử lý ảnh trong các khối DSP.



Hình 3: JPEG được triển khai dưới dạng IP core trên nền tảng FPGA

của thiết bị.

Trong lĩnh vực camera giám sát và hệ thống thị giác máy tính, bộ giải mã JPEG đóng vai trò quan trọng trong việc xử lý luồng ảnh liên tục từ cảm biến hoặc từ mạng truyền dẫn. Các chip xử lý hình ảnh (ISP Image Signal Processor) thường tích hợp sẵn bộ giải mã JPEG để phục vụ việc lưu trữ, truyền tải và phân tích dữ liệu hình ảnh trong thời gian thực.

Ngoài ra, trong các hệ thống nhúng hiệu năng cao và các nền tảng tăng tốc phần cứng, bộ giải mã JPEG có thể được tích hợp như một khối xử lý độc lập kết nối với CPU thông qua bus hệ thống. Cách tiếp cận này cho phép hệ thống mở rộng và tái sử dụng khối giải mã JPEG trong nhiều ứng dụng khác nhau, đồng thời phù hợp với xu hướng thiết kế kiến trúc máy tính hiện đại theo hướng mô-đun hóa.

Thông qua việc nghiên cứu và mô phỏng bộ giải mã JPEG trong báo cáo này, đề tài góp phần làm rõ cách một thuật toán xử lý ảnh phổ biến được hiện thực hóa thành các khối phần cứng cụ thể trong chip, từ đó giúp sinh viên hiểu sâu hơn về mối liên hệ giữa thuật toán, kiến trúc máy tính và ứng dụng thực tế.

1.4 Phạm vi và giới hạn hệ thống

1.4.1 Phạm vi hệ thống

Hệ thống được nghiên cứu, thiết kế và mô phỏng là bộ giải mã ảnh nén JPEG dưới góc nhìn của kiến trúc máy tính. Hệ thống tập trung vào việc giải mã ảnh JPEG theo chuẩn cơ bản (baseline JPEG), phù hợp với các ứng dụng phổ biến và nội dung học phần.

Phạm vi hệ thống bao gồm các khối chức năng chính trong quá trình giải mã ảnh JPEG, cụ thể là:

- Phân tích cấu trúc file JPEG và trích xuất thông tin cần thiết từ phần header.
- Giải mã entropy sử dụng phương pháp Huffman cho dữ liệu DC và AC.
- Khôi phục các hệ số DCT thông qua quá trình giải lượng tử.
- Thực hiện biến đổi cosin ngược hai chiều (IDCT) để chuyển dữ liệu từ miền tần số sang miền không gian.
- Sắp xếp lại các hệ số theo thứ tự zigzag và khôi phục ma trận điểm ảnh khối 8×8 .

- Chuyển đổi không gian màu từ YCbCr sang RGB để phục vụ hiển thị ảnh đầu ra.

Hệ thống được mô phỏng ở mức phần cứng bằng ngôn ngữ mô tả phần cứng, tập trung vào kiến trúc xử lý, luồng dữ liệu và cơ chế điều khiển, phù hợp với nội dung học phần Kiến trúc máy tính.

1.4.2 Giới hạn hệ thống

Do giới hạn về thời gian và phạm vi của học phần, hệ thống chỉ tập trung vào một số khía cạnh nhất định của bộ giải mã JPEG, với một số giới hạn cụ thể:

- Hệ thống chỉ hỗ trợ chuẩn *Progressive JPEG*, không xem xét các chuẩn nén ảnh khác như Baseline JPEG, JPEG-LS hoặc JPEG-2000.
- Quá trình mã hóa ảnh (JPEG encoder) không nằm trong phạm vi nghiên cứu; báo cáo chỉ tập trung vào phía giải mã (decoder).
- Hệ thống được mô phỏng và kiểm chứng chức năng, không thực hiện tổng hợp (synthesis) hay đánh giá chi tiết về diện tích, công suất tiêu thụ và tốc độ tối ưu trên phần cứng thực tế.
- Việc tối ưu hiệu năng ở mức cao như song song hóa sâu, tối ưu pipeline phức tạp hoặc sử dụng các kiến trúc tăng tốc chuyên dụng không được đề cập chi tiết trong phạm vi báo cáo.
- Hệ thống giả định dữ liệu đầu vào là file JPEG hợp lệ, không xử lý các trường hợp lỗi phức tạp hoặc file bị hỏng.

Mặc dù còn tồn tại những giới hạn nhất định, hệ thống được thiết kế vẫn phản ánh đầy đủ các khối chức năng cốt lõi của bộ giải mã JPEG, đồng thời đáp ứng mục tiêu nghiên cứu và yêu cầu của học phần Kiến trúc máy tính.

1.5 Công cụ sử dụng:

Để thực hiện việc thiết kế, mô phỏng và kiểm chứng bộ giải mã ảnh nén JPEG theo chuẩn Progressive JPEG, hệ thống sử dụng các công cụ phần mềm phù hợp với từng giai đoạn của quá trình phát triển hệ thống. Các công cụ được lựa chọn nhằm đảm bảo tính chính xác, khả năng kiểm chứng và phù hợp với nội dung học phần Kiến trúc máy tính.

1.5.1 Ngôn ngữ mô tả phần cứng Verilog

Verilog được sử dụng làm ngôn ngữ mô tả phần cứng chính trong quá trình thiết kế hệ thống. Các khối chức năng của bộ giải mã JPEG như phân tích header, giải mã Huffman, giải lượng tử, biến đổi IDCT, sắp xếp zigzag và chuyển đổi không gian màu được mô tả dưới dạng các module Verilog riêng biệt.

Việc sử dụng Verilog cho phép mô hình hóa chi tiết kiến trúc phần cứng, bao gồm luồng dữ liệu (datapath), luồng điều khiển (control path), các trạng thái hoạt động của hệ thống và cơ chế đồng bộ tín hiệu. Qua đó, hệ thống có thể được kiểm tra và đánh giá ở mức kiến trúc trước khi triển khai trên phần cứng thực tế. [1]

1.5.2 Công cụ mô phỏng Icarus Verilog / ModelSim

Các module Verilog sau khi được thiết kế được mô phỏng và kiểm chứng chức năng bằng các công cụ mô phỏng phần cứng như Icarus Verilog và ModelSim. Icarus Verilog được sử dụng để biên dịch và mô phỏng nhanh các testbench, trong khi ModelSim hỗ trợ quan sát chi tiết dạng sóng tín hiệu (waveform) và kiểm tra hoạt động của hệ thống theo từng chu kỳ xung nhịp.

Thông qua quá trình mô phỏng, các tín hiệu dữ liệu và tín hiệu điều khiển của từng module được theo dõi nhằm đảm bảo hệ thống hoạt động đúng theo thiết kế, đặc biệt đối với các khối xử lý có tính tuần tự và phụ thuộc trạng thái như bộ giải mã Huffman và bộ điều khiển FSM.

1.5.3 Ngôn ngữ Python cho kiểm chứng kết quả

Python được sử dụng như một công cụ hỗ trợ để kiểm chứng tính đúng đắn của kết quả giải mã. Các script Python được xây dựng nhằm đọc dữ liệu đầu ra của hệ thống mô phỏng Verilog, tái tạo ảnh đầu ra và so sánh với kết quả giải mã từ các thư viện chuẩn.

Việc sử dụng Python cho phép kiểm chứng kết quả ở mức hệ thống, bao gồm so sánh giá trị điểm ảnh, đánh giá trực quan chất lượng ảnh và phát hiện sai lệch trong quá trình giải mã. Qua đó, độ chính xác của bộ giải mã JPEG được xác nhận một cách độc lập với quá trình mô phỏng phần cứng.

Sự kết hợp giữa Verilog, công cụ mô phỏng phần cứng và Python kiểm chứng giúp xây dựng một quy trình thiết kế và đánh giá hệ thống chặt chẽ, đảm bảo tính đúng đắn và độ tin cậy của bộ giải mã ảnh nén JPEG.

2 Tổng quan về chuẩn JPEG

2.1 Giới thiệu chuẩn JPEG

JPEG (viết tắt của *Joint Photographic Experts Group*) là một trong những phương pháp nén ảnh có mất mát phổ biến và hiệu quả nhất hiện nay [2]. Kể từ khi ra đời vào năm 1992, dựa trên nền tảng của thuật toán **Biến đổi Cosin rời rạc**, JPEG đã trở thành chuẩn nén hình ảnh thống trị toàn cầu, được ứng dụng rộng rãi trên các thiết bị di động, mạng xã hội và các thiết bị lưu trữ có dung lượng hạn chế.

Đặc điểm cốt lõi của JPEG là khả năng nén dữ liệu với tỷ lệ rất cao, có thể lên tới vài chục lần. Mặc dù ảnh sau khi giải nén sẽ có sự khác biệt so với ảnh gốc và chất lượng suy giảm dần theo hệ số nén, nhưng sự mất mát thông tin này được tính toán dựa trên các nghiên cứu về hệ nhàn thị của mắt người. Bằng cách loại bỏ những chi tiết mà mắt người ít nhạy cảm, JPEG cho phép người dùng lựa chọn sự cân bằng tối ưu giữa dung lượng lưu trữ và chất lượng hình ảnh hiển thị.

Về mặt định dạng, chuẩn JPEG thường được triển khai qua các tệp tin có phần mở rộng như .jpeg, .jfif, .jpg hoặc .jpe, trong đó định dạng .jpg là phổ biến nhất. Trong các kiến trúc máy tính hiện đại, việc thiết kế bộ giải mã JPEG đóng vai trò thiết yếu nhằm xử lý và truyền tải khối lượng hình ảnh khổng lồ hàng ngày, đồng thời tối ưu hóa tài nguyên phần cứng cho các thiết bị kỹ thuật số.



Hình 4: Ảnh chụp một con mèo rừng châu Âu với tỷ lệ nén và các tổn thất liên quan giảm dần từ trái sang phải.

2.2 Nguyên lý nén ảnh JPEG

2.2.1 Chuyển đổi không gian màu

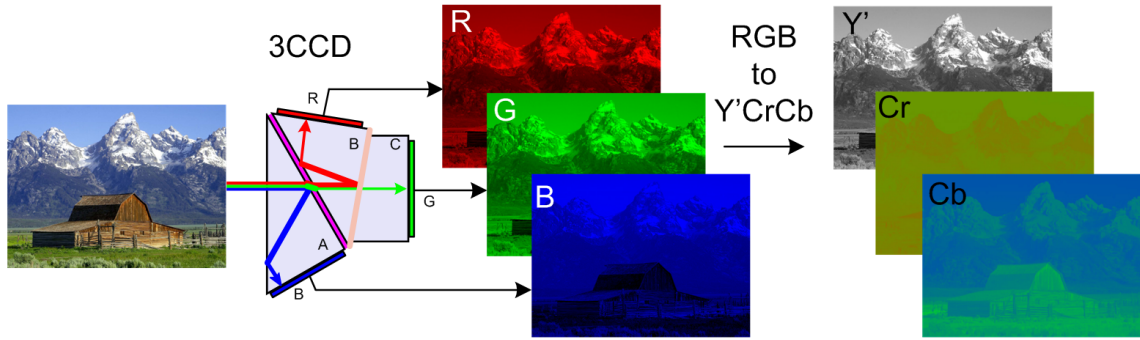
Bước đầu tiên và quan trọng trong quy trình nén **JPEG** là chuyển đổi dữ liệu hình ảnh từ không gian màu RGB sang không gian màu $Y'C_B C_R$.

Không gian màu $Y'C_B C_R$ phân tách hình ảnh thành ba thành phần riêng biệt:

- Y' (**Luma**): Đại diện cho độ sáng của điểm ảnh.
- C_B (**Blue-difference chroma**): Thành phần màu xanh dương.
- C_R (**Red-difference chroma**): Thành phần màu đỏ.

Theo chuẩn JFIF, để đảm bảo tính tương thích tối đa, quá trình chuyển đổi thường tuân theo công thức sau :

$$\begin{cases} Y' &= 0.299R + 0.587G + 0.114B \\ C_B &= -0.1687R - 0.3313G + 0.5B + 128 \\ C_R &= 0.5R - 0.4187G - 0.0813B + 128 \end{cases}$$



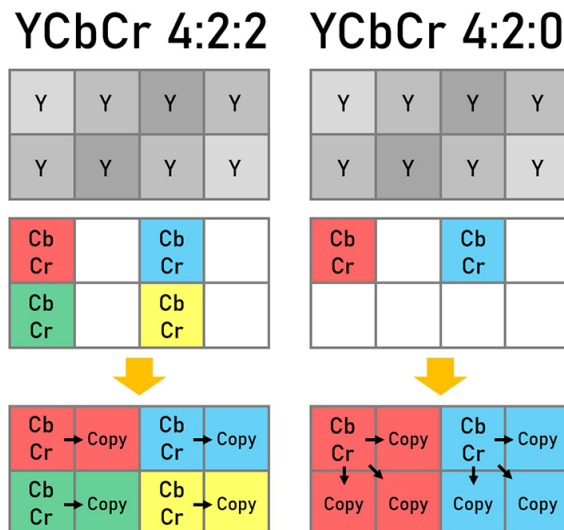
Hình 5: Ảnh minh họa RGB sang YCbCr.

Sau khi chuyển đổi sang không gian màu $Y'CbCr$, để tối ưu hóa dung lượng nén hơn nữa, chuẩn JPEG áp dụng kỹ thuật **lấy mẫu xuống (subsampling)** đối với hai kênh màu C_B và C_R .

Kỹ thuật này dựa trên đặc điểm sinh học của mắt người: nhạy cảm với độ sáng (Y') hơn là chi tiết màu sắc [3]. Do đó, ta có thể giảm độ phân giải của các kênh màu mà không làm ảnh hưởng nhiều đến chất lượng hiển thị.

Trong chuẩn JPEG, tỷ lệ lấy mẫu phổ biến nhất là **4:2:0**.

- **Cơ chế:** Với mỗi khối 2×2 pixel (gồm 4 điểm ảnh), ta giữ nguyên 4 giá trị độ sáng (Y'), nhưng chỉ giữ lại 1 giá trị C_B và 1 giá trị C_R (thường là lấy trung bình hoặc lấy mẫu tại góc).
- **Hiệu quả:** Phương pháp này giúp giảm 50% lượng dữ liệu màu cần xử lý so với ban đầu (tỷ lệ 4:4:4) trước khi đi vào các bước mã hóa phức tạp tiếp theo.



Hình 6: Minh họa sự khác biệt giữa cấu trúc lấy mẫu 4:2:2 và 4:2:0.

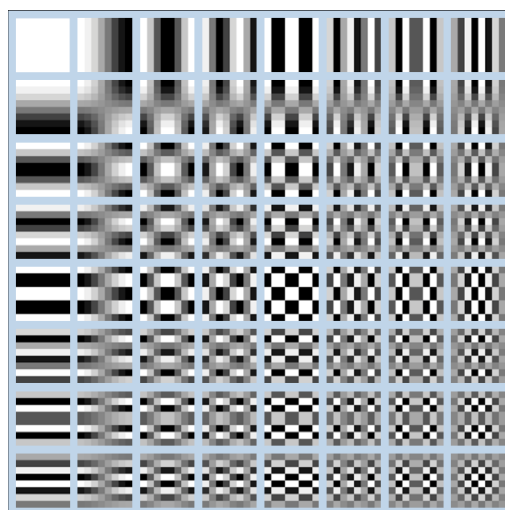
Việc sử dụng tỷ lệ 4:2:0 cũng ảnh hưởng đến định nghĩa của một đơn vị MCU. Thay vì chỉ là một khối 8×8 , một MCU ở chế độ 4:2:0 sẽ bao gồm 4 khối Y (16×16 pixel gốc), 1 khối C_B và 1 khối C_R (tương ứng với vùng không gian đó).

2.2.2 Biến đổi Cosin rời rạc

Sau khi thực hiện biến đổi RGB sang $Y'CB'CR$, với mỗi thành phần $Y'CB'CR$ ta sẽ chia nhỏ hình ảnh đầu vào thành các khối dữ liệu cơ bản có kích thước 8×8 điểm ảnh, được gọi là các **Đơn vị mã hóa tối thiểu (MCUs)**.

Sau đó, thuật toán **Biến đổi Cosin rời rạc (DCT)** sẽ được áp dụng cho từng khối này, và kết quả đầu ra sẽ tiếp tục được đưa qua bước Lượng tử hóa để thực hiện nén dữ liệu.

Về mặt lý thuyết, DCT là phương pháp chuyển đổi tín hiệu từ miền không gian sang miền tần số bằng cách biểu diễn các điểm dữ liệu rời rạc dưới dạng tổng hợp của các sóng cosin. Trong ngữ cảnh của JPEG, thoát nhìn việc chuyển đổi một khối ảnh sang tập hợp các hàm cosin có vẻ phức tạp và không cần thiết. Tuy nhiên, ý nghĩa thực sự của phép biến đổi này nằm ở khả năng **tập trung thông tin**. DCT biến một khối ảnh 8×8 ban đầu thành một ma trận hệ số tần số 8×8 . Ma trận này cho biết mức độ đóng góp của các hàm cosin cơ sở (từ tần số thấp đến tần số cao) để tái tạo lại hình ảnh gốc. Nhờ đó, bước lượng tử hóa tiếp theo có thể loại bỏ các thành phần tần số cao (chi tiết ảnh tinh vi mà mắt người ít nhận thấy) để giảm dung lượng mà không làm ảnh hưởng lớn đến cấu trúc tổng thể của bức ảnh.



Hình 7: Minh họa 64 hàm cơ sở của biến đổi DCT kích thước 8×8 .

2.2.3 Lượng tử hóa

Chúng ta đều đã nghe nói rằng JPEG là một thuật toán nén mất dữ liệu, nhưng cho đến nay chúng ta chưa thực hiện bất kỳ thao tác nén mất dữ liệu nào. Chúng ta chỉ mới chuyển đổi các khối 8×8 thành các khối 8×8 hàm Cosin mà không làm mất thông tin. Phần mất dữ liệu nằm ở bước lượng tử hóa.

Lượng tử hóa là một quá trình trong đó chúng ta lấy một vài giá trị trong một phạm vi cụ thể và biến chúng thành một giá trị rời rạc. Trong trường hợp của chúng ta, đây chỉ là một tên gọi khác cho việc chuyển đổi các hệ số tần số cao hơn trong ma trận đầu ra DCT thành 0. Khi lưu một hình ảnh bằng JPEG, hầu hết các chương trình chỉnh sửa ảnh sẽ hỏi chúng ta cần nén ở mức độ nào. Tỷ lệ phần trăm mà ta cung cấp ở đó sẽ ảnh hưởng đến mức độ lượng tử hóa được áp dụng và lượng thông tin tần số cao hơn bị mất. Đây là nơi mà quá trình nén mất dữ liệu được áp dụng. Một khi đã mất thông tin tần số cao, ta không thể tạo lại chính xác hình ảnh gốc từ hình ảnh JPEG kết quả. [4]

Tùy thuộc vào mức độ nén cần thiết, một số ma trận lượng tử hóa phổ biến được sử dụng (thông tin thú vị: Hầu hết các nhà cung cấp đều có bằng sáng chế về cấu trúc bảng lượng tử hóa). Chúng ta chia ma trận hệ số DCT từng phần tử cho ma trận lượng tử hóa, làm tròn kết quả đến số nguyên, và nhận được ma trận đã lượng tử hóa. Hãy cùng xem một ví dụ ở trang bên.

a) Ma trận các hệ số DCT mẫu:

$$M_{DCT} = \begin{bmatrix} -415 & -33 & -58 & 35 & 58 & -51 & -15 & -12 \\ 5 & -34 & 49 & 18 & 27 & 1 & -5 & 3 \\ -46 & 14 & 80 & -35 & -50 & 19 & 7 & -18 \\ -53 & 21 & 34 & -20 & 2 & 34 & 36 & 12 \\ 9 & -2 & 9 & -5 & -32 & -15 & 45 & 37 \\ -8 & 15 & -16 & 7 & -8 & 11 & 4 & 7 \\ 19 & -28 & -2 & -26 & -2 & 7 & -44 & -21 \\ 18 & 25 & -12 & -44 & 35 & 48 & -37 & -3 \end{bmatrix}$$

b) Ma trận lượng tử hóa:

$$Q = \begin{bmatrix} 16 & 11 & 10 & 16 & 24 & 40 & 51 & 61 \\ 12 & 12 & 14 & 19 & 26 & 58 & 60 & 55 \\ 14 & 13 & 16 & 24 & 40 & 57 & 69 & 56 \\ 14 & 17 & 22 & 29 & 51 & 87 & 80 & 62 \\ 18 & 22 & 37 & 56 & 68 & 109 & 103 & 77 \\ 24 & 35 & 55 & 64 & 81 & 104 & 113 & 92 \\ 49 & 64 & 78 & 87 & 103 & 121 & 120 & 101 \\ 72 & 92 & 95 & 98 & 112 & 100 & 103 & 99 \end{bmatrix}$$

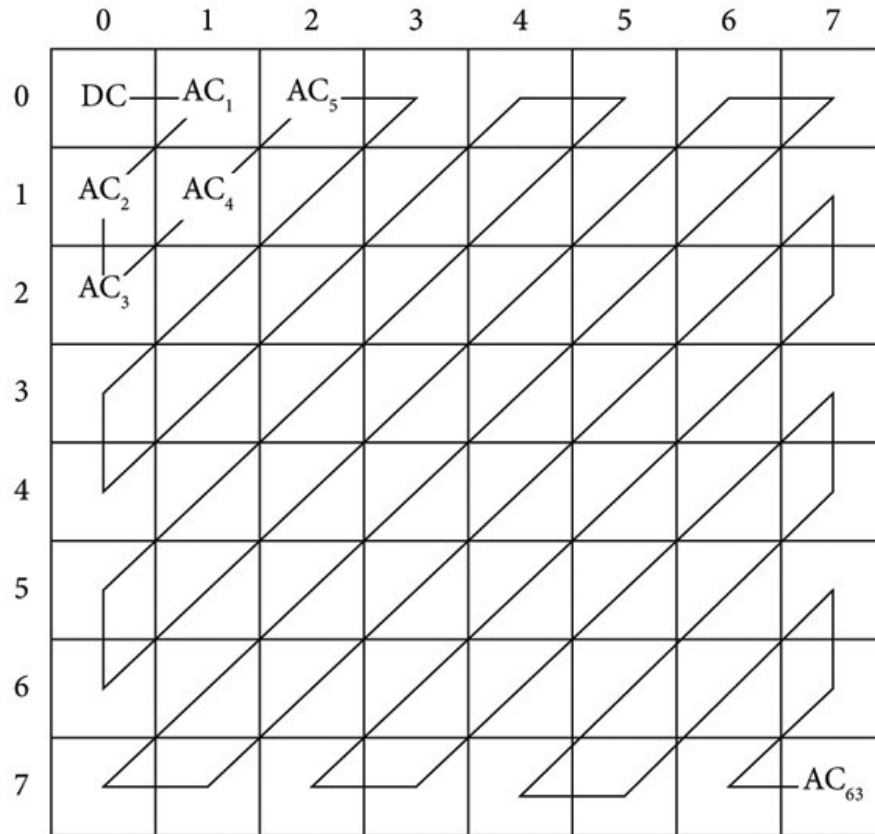
c) Ma trận kết quả sau khi chia và làm tròn:

$$\text{Result} = \text{round} \left(\frac{M_{DCT}}{Q} \right) = \begin{bmatrix} -26 & -3 & -6 & 2 & 2 & -1 & 0 & 0 \\ 0 & -3 & 4 & 1 & 1 & 0 & 0 & 0 \\ -3 & 1 & 5 & -1 & -1 & 0 & 0 & 0 \\ -4 & 1 & 2 & -1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Hình 8: Ví dụ minh họa quá trình lượng tử hóa: Từ các hệ số DCT (a), áp dụng ma trận lượng tử hóa (b) để thu được kết quả nén (c).

2.2.4 Sắp xếp Zig-zag

Sau bước lượng tử hóa, JPEG sử dụng mã hóa zig-zag để chuyển đổi ma trận thành ảnh 1 chiều.



Hình 9: Thứ tự quét zigzag cho khối ảnh 8×8

$$\begin{bmatrix} 15 & 14 & 10 & 9 \\ 13 & 11 & 8 & 0 \\ 12 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Kết quả sau quá trình quét Zig-zag: 15, 13, 14, 12, 11, 10, 0, 0, 8, ... Có thể thấy các giá trị khác 0 tập trung chủ yếu ở góc trên bên trái (nơi chứa các thành phần tần số thấp và DC), trong khi các giá trị ở góc dưới bên phải (tần số cao) đều bằng 0. Điều này thuận lợi cho việc nén RLE sau này.

Sau bước lượng tử hóa và quét Zigzag, các hệ số trong khối 8 × 8 được phân loại thành thành phần một chiều DC và thành phần xoay chiều AC. Do đặc điểm phân bố dữ liệu khác nhau, chuẩn JPEG áp dụng hai kỹ thuật mã hóa riêng biệt trước khi đưa vào nén Huffman. [2]

a, Mã hóa hệ số DC Hệ số DC (hệ số đầu tiên trong mảng Zigzag) đại diện cho độ sáng trung bình của toàn bộ khối ảnh. Do tính chất tương quan không gian của hình ảnh tự nhiên, độ sáng trung bình của các khối liên kề thường thay đổi rất ít.

Thay vì mã hóa trực tiếp giá trị DC do nó có biên độ lớn, chuẩn JPEG sử dụng phương pháp **Mã hóa xung điều biến vi sai**. Giá trị được mã hóa là hiệu số giữa hệ số DC hiện tại và hệ số DC của khối liên trước đó :

$$DIFF = DC_i - DC_{i-1}$$

Trong đó, DC_{i-1} được gọi là giá trị dự đoán. Đối với các kênh màu Y, C_B, C_R , giá trị dự đoán được duy trì độc lập cho từng kênh. Phương pháp này giúp giảm đáng kể dải giá trị cần mã hóa, qua đó tiết kiệm số bit biểu diễn.

b, Mã hóa cho hệ số AC Sau quá trình lượng tử hóa, phần lớn các hệ số AC (đặc biệt là các tần số cao) sẽ bị triệt tiêu về 0. Chuỗi dữ liệu sau khi quét Zigzag thường bao gồm các chuỗi số 0 liên tiếp xen kẽ với các giá trị khác 0.

Để nén hiệu quả chuỗi dữ liệu thừa này, JPEG sử dụng kỹ thuật **Mã hóa độ dài chạy (RLE)**. Thay vì lưu trữ từng số 0 riêng lẻ, hệ thống mã hóa các cặp giá trị bao gồm:

- **Run:** Số lượng các số 0 liên tiếp đứng trước hệ số khác 0 hiện tại (tối đa là 15).
- **Size:** Độ dài của hệ số khác 0 đó.
- **Amplitude:** Giá trị của hệ số đó

Ví dụ: Chuỗi 0, 0, 0, 0, 12 sẽ được mã hóa thành cặp (4, 4)1110.

Hai ký hiệu đặc biệt thường được sử dụng trong RLE của JPEG:

1. **EOB (End of Block):** Ký hiệu kết thúc khối (thường là 0x00), báo hiệu rằng tất cả các hệ số còn lại trong khối Zigzag đều bằng 0.
2. **ZRL (Zero Run Length):** Ký hiệu đại diện cho 16 số 0 liên tiếp (dùng khi chuỗi số 0 dài hơn 15).

Kết quả của quá trình Delta Encoding (cho DC) và RLE (cho AC) sẽ là đầu vào cho bảng mã hóa Huffman để tạo ra chuỗi bit cuối cùng.

2.2.5 Mã hóa Huffman

Mã hóa Huffman là phương pháp nén dữ liệu dựa trên thống kê, hoạt động theo hai nguyên tắc cốt lõi nhằm tối ưu hóa dung lượng lưu trữ và đảm bảo tính chính xác khi giải mã:

1. Tối ưu độ dài từ mã dựa trên tần suất xuất hiện

Thay vì sử dụng số lượng bit cố định cho mọi giá trị (ví dụ: 8 bit cho mỗi ký tự), Huffman sử dụng độ dài bit thay đổi. Nguyên tắc phân phối bit như sau:

- Các giá trị xuất hiện với **tần suất cao** sẽ được gán các từ mã **ngắn** (ít bit).
- Các giá trị xuất hiện với **tần suất thấp** sẽ được gán các từ mã **dài** (nhiều bit).

Chiến lược này giúp giảm thiểu tổng số bit cần thiết để biểu diễn toàn bộ dữ liệu, đặc biệt hiệu quả với dữ liệu ảnh sau biến đổi DCT nơi phần lớn các hệ số là số 0 hoặc các số nhỏ.

2. Đảm bảo tính giải mã duy nhất

Để đảm bảo hệ thống có thể giải mã chính xác chuỗi bit liên tục mà không cần các ký tự ngăn cách, Huffman tạo ra bộ mã có tính chất Không tiền tố.

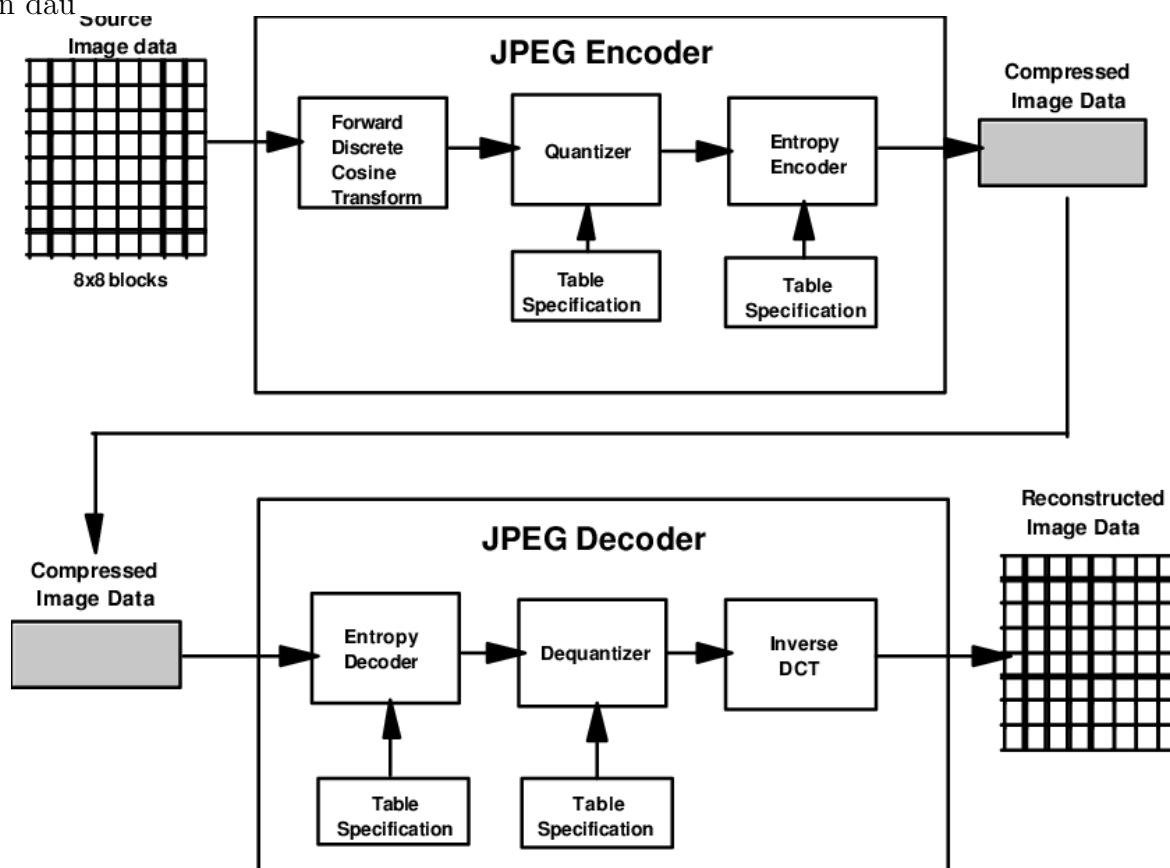
Điều này có nghĩa là: *Không có bất kỳ từ mã nào là phần mở đầu (tiền tố) của một từ mã khác.*

Ví dụ: Nếu mã '0' đã được dùng cho giá trị A, thì không thể có giá trị B nào được mã hóa là '01' hay '001'.

Nhờ tính chất này, quá trình giải mã luôn đảm bảo sự **duy nhất**: ngay khi bộ giải mã tìm thấy một chuỗi bit khớp với bảng mã, nó có thể khẳng định chắc chắn đó là một từ mã hợp lệ và kết thúc việc tìm kiếm cho giá trị đó để chuyển sang giá trị tiếp theo.

2.3 Nguyên lý giải mã JPEG

Sau khi nén ảnh JPEG chúng ta sẽ thực hiện các bước ngược lại để thu được ảnh ban đầu.



Vì vậy trong phần này nhóm em sẽ tìm hiểu về cách các thông tin được lưu trong JPEG. Trong chuẩn nén JPEG, dòng dữ liệu (bitstream) được tổ chức chặt chẽ thông qua các mã đánh dấu gọi là Marker. Việc nhận diện chính xác các Marker này là nhiệm vụ đầu tiên và quan trọng nhất của bộ giải mã để phân tách thông tin header (bảng mã, kích thước ảnh) và dữ liệu nén.

2.3.1 Định nghĩa và Cấu trúc chung của Marker

Trong chuẩn nén JPEG (ISO/IEC 10918-1), luồng dữ liệu được tổ chức thành các phân đoạn (segment) riêng biệt thông qua các mã đánh dấu đặc biệt gọi là **Marker**.

Marker đóng vai trò là các tín hiệu điều khiển, giúp bộ giải mã nhận biết sự thay đổi ngữ cảnh xử lý (ví dụ: chuyển từ đọc thông tin Header sang đọc dữ liệu ảnh, hoặc chuyển từ bảng lượng tử sang bảng Huffman) mà không cần phải giải mã toàn bộ dữ liệu nhị phân.

Cấu trúc tổng quát Hầu hết các Marker trong JPEG (trừ một số trường hợp đặc biệt) đều tuân theo cấu trúc gói dữ liệu gồm 3 phần chính như sau:

$$\text{Segment} = \underbrace{0xFF \ || \ ID}_{\text{Marker (2 bytes)}} + \underbrace{Length}_{2 \text{ bytes}} + \underbrace{Payload Data}_{(\text{Length} - 2) \text{ bytes}}$$

Trong đó:

- **Marker Prefix (Tiền tố Marker):** Luôn là byte `0xFF`. Đây là dấu hiệu nhận biết để bộ tách dòng bit (Bitstream Parser) phát hiện sự hiện diện của một Marker. [4]
- **Marker ID (Mã định danh):** Là byte xác định loại Marker (ví dụ: `0xC0` là SOF0, `0xC4` là DHT). Byte này luôn khác `0x00` và khác `0xFF`.
- **Length (Độ dài phân đoạn):** Là một số nguyên 16-bit (Big-endian) chỉ ra tổng độ dài của tham số theo sau.
 - Giá trị Length bao gồm cả 2 byte của chính nó nhưng **không** bao gồm 2 byte của Marker.
 - Công thức tính lượng dữ liệu thực tế cần đọc: $N_{data} = \text{Length} - 2$.
- **Payload Data (Dữ liệu tải):** Chứa các thông tin cấu hình (như nội dung bảng lượng tử, bảng Huffman, kích thước ảnh...).

Một số Marker đặc biệt không chứa trường *Length* và không có dữ liệu đi kèm. Chúng chỉ đóng vai trò như một cờ hiệu kích hoạt trạng thái. Các Marker này bao gồm:

- **SOI (Start of Image - 0xFFD8):** Báo hiệu bắt đầu file.
- **EOI (End of Image - 0xFFD9):** Báo hiệu kết thúc file.
- **RSTn (Restart Marker - 0xFFD0 đến 0xFFD7):** Dùng để đồng bộ lại quá trình giải mã Entropy (thường dùng khi truyền dữ liệu qua môi trường dễ lỗi).

2.3.2 Một số Marker quan trọng

Tên Marker	Mã Hex	Chức năng và Ý nghĩa
SOI	0xFFD8	Start of Image: Báo hiệu bắt đầu tệp ảnh. Kích hoạt trạng thái khởi tạo của bộ giải mã.
APPn	0xFFE0-EF	Application Data: Chứa thông tin phụ trợ (JFIF, Exif...). Bộ giải mã phần cứng thường bỏ qua (skip) các phân đoạn này dựa trên trường Length.
DQT	0xFFDB	Define Quantization Table: Định nghĩa một hoặc nhiều bảng lượng tử (8×8). Cần thiết cho bước giải lượng tử.
SOF0	0xFFC0	Start of Frame (Baseline): Chứa các thông số kỹ thuật quan trọng nhất: chiều rộng, chiều cao ảnh, và tỷ lệ lấy mẫu (Subsampling 4:2:0, 4:2:2...).
DHT	0xFFC4	Define Huffman Table: Chứa dữ liệu để xây dựng bảng Huffman (cho DC và AC).
DRI	0xFFDD	Define Restart Interval: (Tùy chọn) Xác định khoảng cách giữa các lần reset bộ giải mã Entropy (dùng cho cơ chế sửa lỗi).
SOS	0xFFDA	Start of Scan: Đánh dấu kết thúc phần Header. Ngay sau marker này là luồng dữ liệu nén Entropy (Scan Data) cần được giải mã.
EOI	0xFFD9	End of Image: Báo hiệu kết thúc tệp ảnh.

Bảng 1: Danh sách các Marker quan trọng trong chuẩn nén JPEG

3 Kiến trúc tổng thể bộ giải mã JPEG

3.1 Cấu trúc phân cấp và Tổ chức các Module

Dựa trên kiến trúc tổng thể đã trình bày, hệ thống giải mã JPEG được hiện thực hóa trên phần cứng thông qua phương pháp thiết kế phân cấp. Các chức năng phức tạp của thuật toán giải mã Baseline JPEG được chia nhỏ thành các module xử lý độc lập, giúp tối ưu hóa quá trình tổng hợp và kiểm thử.

Toàn bộ mã nguồn RTL (Register-Transfer Level) được tổ chức thành các tầng xử lý chuyên biệt trong thư mục dự án, đảm bảo tính mạch lạc trong luồng dữ liệu từ khi nhận byte dữ liệu nén cho đến khi xuất ra pixel màu RGB. Sơ đồ dưới đây mô tả cấu trúc cây thư mục và vai trò của từng thành phần trong hệ thống:

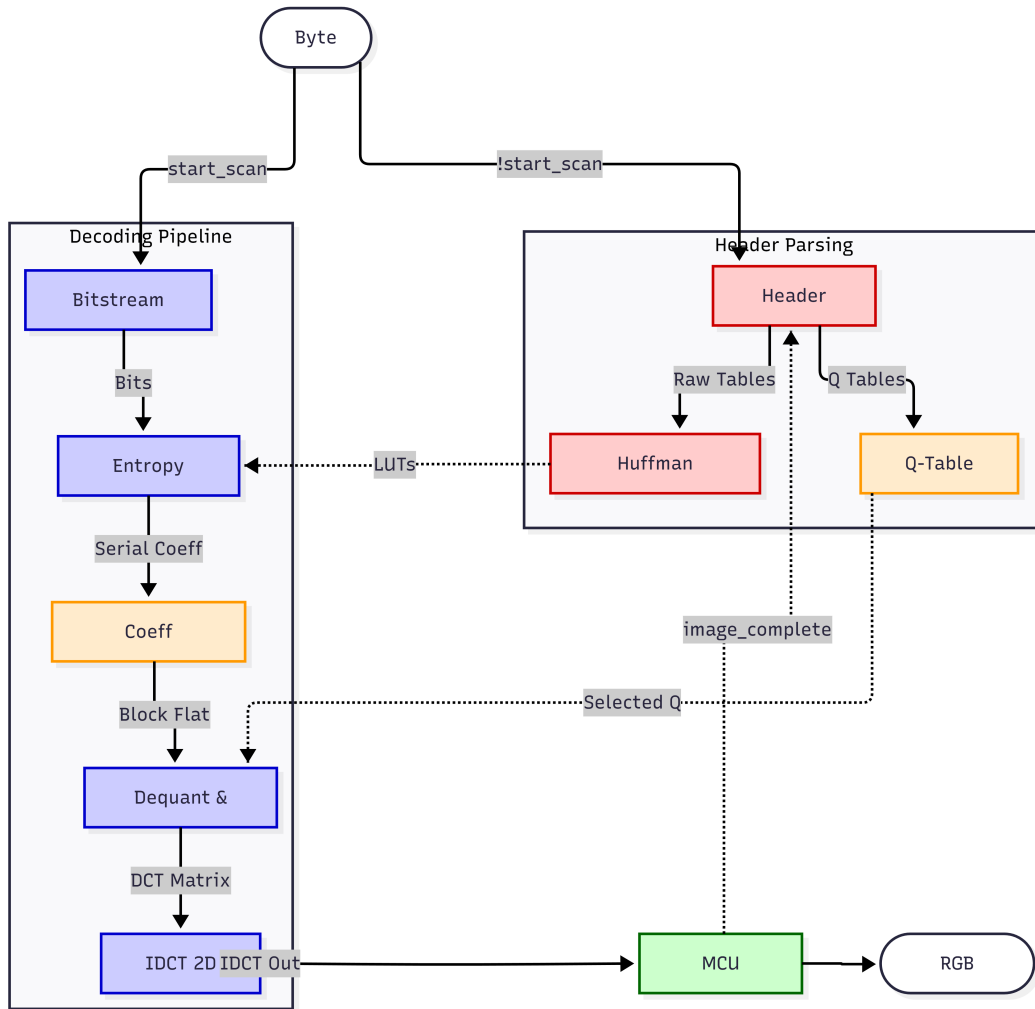


Hình 10: Kiến trúc phân cấp và tổ chức các module RTL của hệ thống

3.2 Thiết kế Module Top-level và Logic điều phối

Module `jpeg_decoder_top.v` đóng vai trò là "bộ não" điều khiển toàn bộ quá trình giải mã. Thay vì trực tiếp xử lý dữ liệu, module này thực hiện việc quản lý các kết nối phức tạp giữa các khối chức năng và điều phối tài nguyên dùng chung.

Sơ đồ dưới đây mô tả luồng dữ liệu chính và các module chức năng trong file `jpeg_decoder_top.v`. Dữ liệu đầu vào là các byte ảnh nén JPEG và đầu ra là luồng pixel RGB.



Hình 11: Sơ đồ khối kiến trúc bộ giải mã JPEG (tổng hợp từ module Top)

3.2.1 Quản lý "Biến chung" và Kết nối liên tầng

Trong thiết kế phần cứng của hệ thống, Module Top quản lý các bus dữ liệu có độ rộng cực lớn, đóng vai trò là các biến dùng chung để các module trao đổi thông tin trong cùng một chu kỳ xung nhịp:

- **Tín hiệu điều khiển toàn cục (`start_scan`):** Đây là biến trạng thái quan trọng nhất được truyền từ khối `u_parser` đến các khối xử lý phía sau. Nó đóng vai trò là lệnh "kích hoạt" để chuyển toàn bộ hệ thống từ trạng thái nạp cấu hình sang trạng thái giải mã ảnh thực tế.

- **Phẳng hóa và mở rộng Bus dữ liệu (Bus Flattening):** Để tối ưu hóa tốc độ, Module Top thực hiện kỹ thuật phẳng hóa các mảng dữ liệu thành các bus đơn có độ rộng lớn:
 - **Bảng lượng tử (q_16):** Module Top sử dụng khối `generate` để mở rộng 64 hệ số 8-bit từ bảng lượng tử thành một bus 1024-bit. Kỹ thuật này cho phép khối `u_trans` nhận toàn bộ bảng thông số trong một nhịp xử lý duy nhất.
 - **Bus truyền khối (blk_flat):** Một bus 1024-bit kết nối từ bộ giải mã Entropy sang khối Transform, cho phép truyền tải đồng thời 64 hệ số DCT đã giải mã mà không cần cơ chế truyền nhận tuần tự phức tạp.

3.2.2 Logic điều phối Pipeline và Đồng bộ hóa nội bộ

Module Top đảm bảo tính toàn vẹn dữ liệu thông qua các cơ chế điều phối logic sau:

- **Logic dồn kênh đầu vào:** Tín hiệu phản hồi `byte_ready` gửi ra ngoài được Module Top quyết định dựa trên trạng thái của biến `start_scan`. Điều này giúp che giấu sự phức tạp bên trong, khiến môi trường bên ngoài thấy bộ giải mã như một thực thể duy nhất:

$$\text{byte_ready} = \text{start_scan} ? \text{bs_byte_ready} : \text{parser_ready}$$

- **Cơ chế chốt dữ liệu:** Một điểm kỹ thuật đặc biệt trong mã nguồn là việc xử lý lệch pha Pipeline. Module Top thực hiện chốt bảng lượng tử (`q_table_latched`) chính xác khi tín hiệu `blk_done` kích hoạt. Việc này đảm bảo bảng lượng tử được áp dụng đúng cho khối dữ liệu đang nằm trong hàng đợi xử lý, tránh hiện tượng sai lệch màu sắc do dữ liệu mới ghi đè dữ liệu cũ.
- **Lựa chọn bảng theo thành phần (use_tb1):** Dựa trên bộ đếm khối (`blk_cnt_mux`), Module Top điều khiển việc chọn lựa giữa bảng Luma (bảng 0) và bảng Chroma (bảng 1) để cấp cho bộ giải mã Entropy, đảm bảo tính nhất quán của chuẩn JPEG Baseline.

3.2.3 Bảng tóm tắt vai trò và Biến giao tiếp giữa các file

Bảng dưới đây mô tả cách Module Top phân chia công việc cho các file lẻ và các biến chung dùng để kết nối chúng:

Thực thể (Instance)	Vai trò chuyên trách	Biến chung kết nối
<code>u_parser</code>	Trích xuất siêu dữ liệu	<code>q_table</code> , <code>huff_table</code> , <code>start_scan</code>
<code>u_bs</code>	Cung cấp luồng bit sạch	<code>bit_stream</code> , <code>bit_valid</code>
<code>u_ent</code>	Giải mã ký hiệu Huffman	<code>blk_flat</code> , <code>blk_done</code> , <code>use_tb1</code>
<code>u_trans</code>	Giải lượng tử và Zig-Zag	<code>q_16</code> , <code>dct_flat</code>
<code>u_idct</code>	Biến đổi ngược Cosine	<code>idct_in</code> , <code>idct_out_flat</code>
<code>u_mcu</code>	Quản lý đơn vị MCU & RGB	<code>r_out</code> , <code>g_out</code> , <code>b_out</code> , <code>pixel_valid</code>

Bảng 2: Phân tách vai trò và các biến giao tiếp chính trong Module Top

4 Thiết kế chi tiết từng khối chức năng

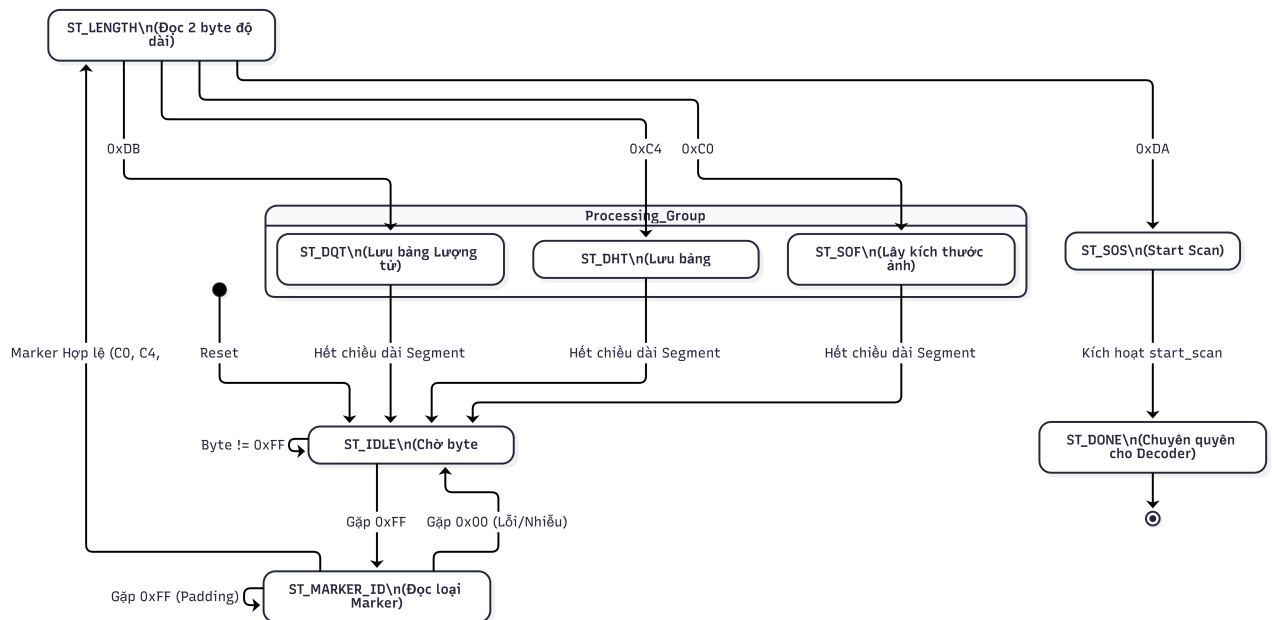
4.1 Byte Stream Parser & Marker Decoder

Khối `jpeg_header_parser` đóng vai trò là "bộ não" trong giai đoạn khởi tạo. Nó chịu trách nhiệm quét luồng byte đầu vào, loại bỏ các dữ liệu không cần thiết (Metadata, Thumbnail) và trích xuất các bảng cấu hình quan trọng để thiết lập cho chuỗi giải mã.

4.1.1 Máy trạng thái quản lý Marker (FSM Control)

Hoạt động của khối được điều khiển bởi một máy trạng thái hữu hạn (FSM) với logic chuyển đổi chặt chẽ để đảm bảo đồng bộ với cấu trúc file JPEG. Quá trình quét được chia thành các giai đoạn chính:

- **Phát hiện Marker (ST_IDLE → ST_MARKER_ID):** Theo chuẩn JPEG, mọi marker đều bắt đầu bằng byte `0xFF`. FSM duy trì trạng thái chờ tại `ST_IDLE`. Khi phát hiện `0xFF`, nó chuyển sang trạng thái `ST_MARKER_ID` để đọc byte tiếp theo (ví dụ: `0xC0`, `0xDB`). Nếu byte tiếp theo là `0x00` hoặc `0xFF` (padding), FSM sẽ quay lại hoặc giữ nguyên trạng thái chờ để lọc nhiễu.
- **Đọc độ dài Segment (ST_LENGTH_HI/LO):** Ngay sau byte định danh marker là 2 byte chỉ định độ dài của segment đó. Parser đọc giá trị này vào thanh ghi `length_cnt` để biết cần đọc (hoặc bỏ qua) bao nhiêu byte tiếp theo.



Hình 12: Sơ đồ Máy trạng thái FSM cho `jpegheaderparser`

4.1.2 Xử lý các Segment quan trọng

- **Xử lý Bảng Lượng tử - DQT (0xDB):** Khi gặp marker `0xDB`, FSM chuyển sang trạng thái `ST_DQT_READ`.

- Hệ thống hỗ trợ đọc nhiều bảng lượng tử (ID 0 hoặc 1) trong cùng một segment.
 - 64 byte dữ liệu của mỗi bảng được lưu vào bộ nhớ đệm `qtable_mem[ID][0..63]`.
 - *Cơ chế Output*: Để tối ưu hóa tốc độ truy xuất cho khối Dequantizer, mảng 2 chiều này được "trải phẳng" (flatten) thành bus dữ liệu rộng `q_quant_table_flat` (512-bit) ngay trong phần logic tổ hợp (Combinational Logic).
- **Xử lý Bảng Huffman - DHT (0xC4)**: Quá trình trích xuất bảng Huffman phức tạp hơn và được chia làm 2 bước trong FSM:
 1. **Đọc số lượng mã (ST_DHT_COUNTS)**: Đọc 16 byte đầu tiên để xác định có bao nhiêu mã Huffman cho mỗi độ dài bit (từ 1 đến 16 bit). Dữ liệu này được lưu vào các mảng `len_out`.
 2. **Đọc giá trị Symbol (ST_DHT_SYMBOLS)**: Dựa trên tổng số lượng mã đã tính toán, FSM tiếp tục đọc các byte symbol và lưu vào các mảng `val_out`.

Hệ thống phân loại tự động dựa trên thông tin Class (DC/AC) và ID để lưu vào đúng thanh ghi mục tiêu (`dc0`, `ac1`,...).

- **Xử lý Start of Frame - SOF0 (0xC0)**: Đây là nơi chứa metadata của ảnh. FSM lần lượt trích xuất:
 - `img_height`, `img_width`: Kích thước ảnh.
 - `comp_h_samp`, `comp_v_samp`: Hệ số lấy mẫu (Sampling Factors) để xác định định dạng màu (4:4:4 hay 4:2:0).
 - `comp_quant_id`: Xác định kênh màu nào sử dụng bảng lượng tử nào.
- **Xử lý Start of Scan - SOS (0xDA)**: Marker này đánh dấu điểm kết thúc của Header. Khi phát hiện `0xDA`, FSM thực hiện bước chuyển giao quyền lực (Handover):
 - Kích hoạt tín hiệu `start_scan = 1`.
 - Vô hiệu hóa tín hiệu `parser_ready = 0`.
 - Máy trạng thái chuyển sang `ST_DONE` và "ngủ" cho đến khi hệ thống được Reset.

4.2 Bitstream Reader & Byte Stuffing

Sau khi Parser hoàn tất nhiệm vụ, module `jpeg_bitstream_reader` nắm quyền kiểm soát bus dữ liệu. Nhiệm vụ của nó là chuyển đổi luồng byte song song (8-bit) thành luồng bit nối tiếp (Serial Bitstream) và loại bỏ các byte giả (Stuffing bytes).

4.2.1 Cơ chế loại bỏ Byte Stuffing (0xFF00)

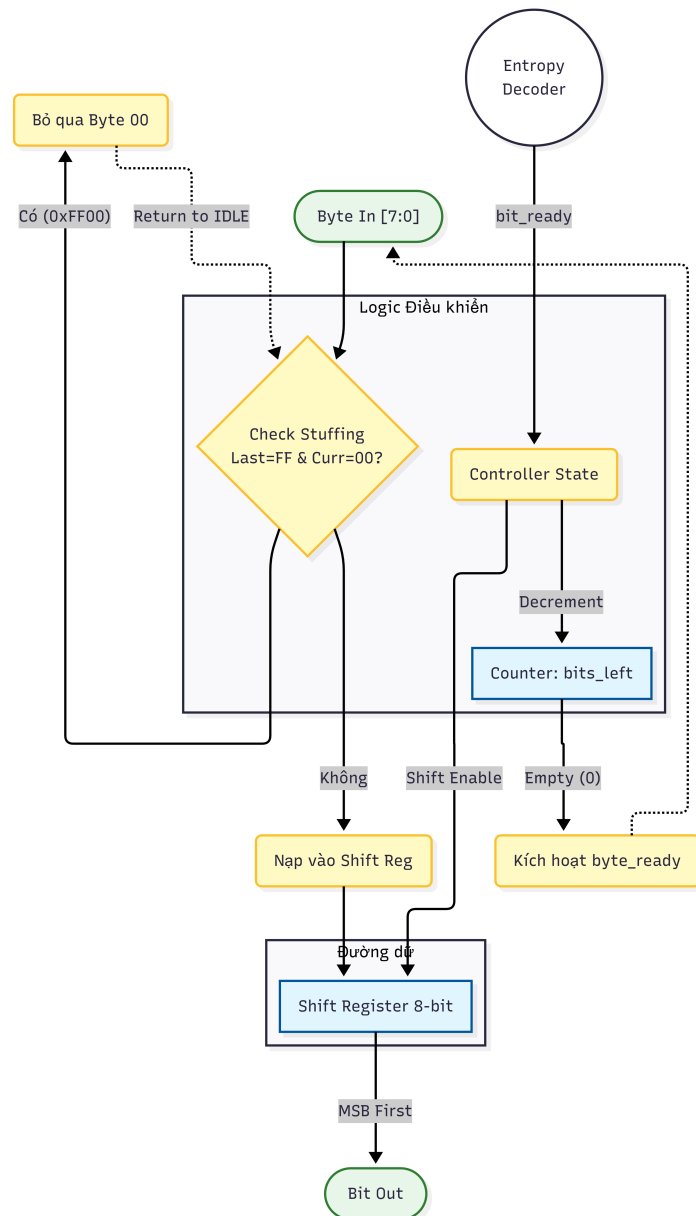
Trong chuẩn JPEG, byte `0xFF` có thể xuất hiện trong dữ liệu nén. Để tránh nhầm lẫn với Marker, encoder sẽ chèn thêm một byte `0x00` ngay sau `0xFF`. Bên phía Decoder, module Reader phải phát hiện và loại bỏ byte này.

Logic này được thực hiện trong trạng thái `S_READ_BYTE`:

```

if (last_was_ff && byte_in == 8'h00) begin
    last_was_ff <= 0;
    state <= S_IDLE; // Bỏ qua, không nạp vào Shift Register
end else begin
    shift_reg <= byte_in; // Nạp dữ liệu hợp lệ
    // ...
end

```



Hình 13: Sơ đồ luồng dữ liệu (Datapath) xử lý Byte Stuffing và Shifting

- **Parallel-to-Serial Converter:** Dữ liệu được nạp vào thanh ghi `shift_reg` [7:0]. Biến đếm `bits_left` theo dõi số bit còn lại hợp lệ trong thanh ghi.

- **Cơ chế Demand-Driven (Theo nhu cầu):** Module hoạt động thụ động dựa trên tín hiệu yêu cầu `bit_ready` từ khối Entropy Decoder phía sau.
 - Nếu `bit_ready = 1` và `bits_left > 0`: Module dịch 1 bit ra cổng `bit_out` (MSB first) và giảm biến đếm.
 - Nếu `bits_left = 0`: Module tạm dừng cấp bit, kích hoạt `byte_ready = 1` để yêu cầu byte mới từ bộ nhớ ngoài.
- **Tín hiệu Output tổ hợp:** Tín hiệu `bit_valid` được gán bằng logic tổ hợp (combinational logic) thay vì thanh ghi tuần tự để giảm độ trễ (latency) giao tiếp với bộ giải mã Huffman:

```
assign bit_valid = (state == S_SHIFT && bits_left > 0); (1)
```

4.3 Bộ giải mã Entropy

Khối giải mã Entropy là thành phần trung tâm trong hệ thống giải mã JPEG, thực hiện nhiệm vụ chuyển đổi luồng bit nén thành các hệ số DCT thô (Quantized DCT coefficients).

4.3.1 Cấu trúc bảng Huffman

Trong thiết kế phần cứng này, bảng Huffman không được lưu trữ dưới dạng cấu trúc cây truyền thống do giới hạn về tốc độ truy xuất. Thay vào đó, hệ thống sử dụng cơ chế nạp bảng dựa trên mảng tra cứu:

- **Khởi tạo bảng:** Module `jpeg_huffman_generator` nhận dữ liệu từ Header Parser bao gồm 16 byte độ dài (`huff_count`) và danh sách các giá trị (`huff_val`). Từ đó, module tự động tính toán ra các mã Huffman (`huff_code`) và độ dài mã tương ứng (`huff_len`).
- **Lưu trữ song song:** Hệ thống duy trì đồng thời 4 cặp bảng: DC0, AC0 (cho thành phần Luma) và DC1, AC1 (cho thành phần Chroma). Tín hiệu `use_tb1` trong module Top được sử dụng để lựa chọn bảng mã phù hợp tùy theo loại block đang được xử lý (Y, Cb hoặc Cr).
- **Cơ chế nạp:** Việc tạo mã chỉ bắt đầu khi tín hiệu `start_scan` được kích hoạt, đảm bảo toàn bộ dữ liệu từ Header đã được nạp xong vào các mảng trung gian. [4]

4.3.2 Thuật toán so khớp bit

Thay vì đọc từng bit đơn lẻ, hệ thống sử dụng cơ chế so khớp song song để đạt hiệu năng cao:

- **Cửa sổ bit:** Module `jpeg_bitstream_reader` cung cấp luồng bit thông qua `bit_out` và `bit_valid`. Dữ liệu được đưa vào một thanh ghi dịch để tạo ra một "cửa sổ" bit liên tục phục vụ cho việc so sánh.
- **So khớp song song:** Module `jpeg_huffman_decoder` thực hiện so sánh đoạn bit hiện tại với tất cả các mã lưu trữ trong bảng tra cứu đồng thời. Khi phát hiện một mã khớp (*match*), hệ thống ngay lập tức trích xuất được Symbol tương ứng và độ dài của mã đó.

- **Cập nhật Bitstream:** Ngay sau khi một mã được giải xong, tín hiệu `bit_ready` được gửi ngược lại bộ đọc Bitstream để dịch bỏ đúng số bit đã sử dụng, chuẩn bị cho chu kỳ giải mã tiếp theo.

4.3.3 Xử lý hệ số DC / AC

Quy trình giải mã được điều khiển chặt chẽ bởi máy trạng thái để phân biệt giữa hệ số DC và các hệ số AC:

- **Hệ số DC:** Là hệ số đầu tiên của mỗi khối. Sau khi giải mã Huffman để xác định nhóm kích thước, hệ thống sẽ đọc thêm các bit bổ sung để xác định giá trị sai lệch (Diff). Giá trị DC cuối cùng được tính toán và lưu vào vị trí đầu tiên của mảng hệ số.
- **Hệ số AC:** Gồm 63 hệ số tiếp theo. Mỗi mã Huffman AC giải mã ra cặp giá trị Run-Length (số lượng hệ số 0 đứng trước) và Size (số bit của giá trị hệ số khác 0).
- **Kết thúc khối (EOB):** Khi gặp mã End of Block (0x00), hệ thống sẽ kích hoạt tín hiệu `block_done`, tự động điền các giá trị 0 cho phần còn lại của khối 8×8 và chuyển sang khối tiếp theo.

4.3.4 So sánh với thuật toán phần mềm

Việc triển khai Huffman Decoder trên FPGA/ASIC mang lại sự khác biệt rõ rệt so với môi trường phần mềm:

- **Tốc độ xử lý:** Trong phần mềm, việc duyệt cây Huffman mất nhiều chu kỳ lệnh tỉ lệ thuận với độ dài mã. Trong phần cứng, nhờ logic tổ hợp và mảng tra cứu song song, việc tìm ra symbol và độ dài mã có thể thực hiện gần như trong một vài nhịp xung nhịp.
- **Xử lý mức Bit:** Phần cứng xử lý việc trích xuất bit không căn lề từ luồng byte hiệu quả hơn rất nhiều so với các thao tác dịch bit phức tạp trên CPU thông qua module `jpeg_bitstream_reader` chuyên biệt.
- **Quản lý MCU:** Việc chuyển đổi giữa các bảng mã (Luma/Chroma) và đếm số lượng block trong một đơn vị MCU được thực hiện tự động bằng phần cứng thông qua tín hiệu `blk_cnt_mux`, giúp giảm tải hoàn toàn cho bộ vi xử lý điều khiển.

4.4 Giải mã hệ số & Dequantization

Sau khi giải mã Huffman, các hệ số nhận được vẫn ở dạng lượng tử hóa và cần được đưa qua quá trình giải mã ngược lượng tử để khôi phục lại biên độ gần đúng của các thành phần tần số DCT.

4.4.1 Bảng lượng tử

Trong thiết kế phần cứng này, bảng lượng tử được quản lý và tổ chức để tối ưu hóa việc truy xuất song song:

- **Lưu trữ và Truy xuất:** Các giá trị bảng lượng tử (64 byte mỗi bảng) được Header Parser trích xuất từ marker DQT và lưu vào bộ nhớ nội bộ `qtable_mem`.
- **Cấu trúc Phẳng hóa:** Để thuận tiện cho việc truyền tải giữa các module, mảng 2 chiều được chuyển đổi thành các vector phẳng 512-bit (`q_quant_table_flat` và `q_quant_table_1_flat`).
- **Phân loại bảng:** Hệ thống sử dụng hai bảng riêng biệt: Bảng 0 dành cho thành phần Luma (Y) và Bảng 1 dành cho các thành phần Chroma (Cb, Cr). Việc lựa chọn bảng được điều phối bởi tín hiệu `use_tb1` đồng bộ với quá trình giải mã entropy.
- **Chốt dữ liệu:** Để đảm bảo tính đồng bộ khi dữ liệu đi qua pipeline, bảng lượng tử được chốt lại (`q_table_latched`) ngay khi một khối 8×8 hoàn tất giải mã (`blk_done`), giúp căn lề chính xác dữ liệu bảng lượng tử với các hệ số tương ứng.

4.4.2 Phép nhân ngược lượng tử

Quá trình khôi phục giá trị hệ số DCT được thực hiện thông qua phép nhân ma trận giữa hệ số sau giải mã và bảng lượng tử tương ứng:

- **Nguyên lý thực hiện:** Mỗi hệ số $C(u, v)$ sau khi giải mã Huffman được nhân với giá trị lượng tử tương ứng $Q(u, v)$ từ bảng tra cứu để tạo ra hệ số DCT khôi phục: $F(u, v) = C(u, v) \times Q(u, v)$.
- **Định dạng dữ liệu:** Trước khi nhân, các giá trị 8-bit từ bảng lượng tử được mở rộng lên 16-bit (`q_16`) để đảm bảo độ chính xác và tránh tràn số khi thực hiện phép tính với các hệ số DCT có dấu.
- **Xử lý song song:** Trong module `jpeg_transform_block`, quá trình dequantization được thực hiện song song trên toàn bộ vector phẳng của khối 8×8 . Việc này cho phép hệ thống cung cấp toàn bộ 64 hệ số đã giải mã ngược lượng tử cho khối IDCT trong cùng một nhịp xử lý.
- **Kết nối hệ thống:** Dữ liệu sau nhân được chuyển đổi từ dạng phẳng (`dct_out_flat`) sang mảng các hệ số 16-bit có dấu (`idct_in`) để sẵn sàng cho quá trình biến đổi ngược DCT 2 chiều.

4.5 Xấp xếp Zig-Zag

Quá trình quét Zig-Zag là một bước quan trọng để sắp xếp lại các hệ số DCT từ chuỗi tuyến tính trở lại vị trí ma trận 8×8 theo đúng thứ tự tần số từ thấp đến cao.

4.5.1 Mapping chỉ số

Trong chuẩn JPEG, dữ liệu sau giải nén Huffman được sắp xếp theo thứ tự Zig-Zag để tập trung các hệ số có giá trị lớn (tần số thấp) ở đầu chuỗi và các hệ số bằng 0 (tần số cao) ở cuối chuỗi.

- **Quy luật quét:** Quá trình này bắt đầu từ hệ số DC (vị trí [0,0]), sau đó quét theo đường chéo để chuyển đổi mảng 1 chiều 64 phần tử thành ma trận 2 chiều 8×8 .
- **Bảng ánh xạ:** Trong phần cứng, quy luật này được thực hiện thông qua một bảng ánh xạ chỉ số cố định. Ví dụ: chỉ số thứ 1 trong mảng 1 chiều được đưa về vị trí (0,1) trong ma trận, chỉ số thứ 2 về (1,0), chỉ số thứ 3 về (2,0), và tiếp tục cho đến chỉ số thứ 63.
- **Khôi phục tần số:** Việc tái cấu trúc này đảm bảo các hệ số DCT được đặt đúng vào các hàm cơ sở tần số tương ứng, chuẩn bị cho bước biến đổi ngược DCT (IDCT).

4.5.2 Cài đặt phần cứng

Để đạt hiệu năng cao và tối ưu hóa tài nguyên trên FPGA, việc quét Zig-Zag trong thiết kế này không sử dụng các vòng lặp tuần tự mà được triển khai bằng logic tổ hợp và đánh địa chỉ song song.

- **Cấu trúc không dây:** Thay vì sử dụng bộ nhớ RAM và đọc từng địa chỉ, module `jpeg_inverse_zigzag` (được tích hợp trong `jpeg_transform_block`) sử dụng các lệnh gán liên tục (`assign`) hoặc kết nối dây trực tiếp. Mỗi vị trí của vector đầu vào được nối thẳng tới vị trí đích tương ứng trong vector đầu ra.
- **Xử lý theo khối:** Nhờ cách tiếp cận kết nối song song, toàn bộ 64 hệ số của một khối 8×8 được sắp xếp lại chỉ trong một chu kỳ xung nhịp. Điều này loại bỏ hoàn toàn độ trễ so với việc sử dụng bộ vi xử lý để thực hiện các phép tính chỉ số phức tạp.
- **Tích hợp Pipeline:** Kết quả của quá trình Zig-Zag (`zz_in_flat`) được đưa trực tiếp vào bộ nhân lượng tử và khối IDCT. Việc tối ưu hóa đánh địa chỉ này giúp duy trì luồng dữ liệu liên tục, đảm bảo rằng hệ thống có thể xử lý một khối dữ liệu mới ngay khi khối trước đó vừa hoàn tất.

4.6 IDCT (Inverse Discrete Cosine Transform)

Đây là khối chức năng quan trọng nhất trong việc giải mã, có nhiệm vụ chuyển đổi dữ liệu từ miền tần số trở về miền không gian.

4.6.1 Cơ sở lý thuyết

Công thức IDCT 2D chuẩn cho một khối 8×8 được định nghĩa như sau:

$$f(x, y) = \frac{1}{4} \sum_{u=0}^7 \sum_{v=0}^7 \alpha(u) \alpha(v) F(u, v) \cos \left[\frac{(2x+1)u\pi}{16} \right] \cos \left[\frac{(2y+1)v\pi}{16} \right]$$

[3] Trong đó:

- $F(u, v)$: Các hệ số DCT trong miền tần số (đầu vào).
- $f(x, y)$: Giá trị điểm ảnh được khôi phục (đầu ra).
- $\alpha(k)$: Hệ số chuẩn hóa, với $\alpha(k) = \frac{1}{\sqrt{2}}$ nếu $k = 0$, và $\alpha(k) = 1$ nếu $k > 0$.

Việc tính toán trực tiếp công thức trên đòi hỏi độ phức tạp $O(N^4)$, tiêu tốn nhiều tài nguyên phần cứng. Tuy nhiên, nhờ tính chất tách biệt của hàm Cosine, công thức có thể được phân tách thành hai bước biến đổi 1D liên tiếp:

$$\text{IDCT}_{2D} = \text{IDCT}_{1D_col}(\text{IDCT}_{1D_row}(F))$$

4.6.2 Kiến trúc phần cứng

Dựa trên nguyên lý trên, module `jpeg_idct_2d.v` được thiết kế theo mô hình đường ống bao gồm 4 giai đoạn chính:

1. **IDCT 1D theo Hàng (Row Processing)**: Xử lý song song 8 hàng của khối dữ liệu đầu vào. Mỗi hàng sử dụng một thực thể (instance) của module `jpeg_idct_1d`.
2. **Bộ đệm chuyển vị (Transpose Buffer)**: Kết quả sau khi biến đổi hàng được lưu vào mảng các thanh ghi và thực hiện hoán vị chỉ số: hàng i , cột j trở thành hàng j , cột i .
3. **IDCT 1D theo Cột (Column Processing)**: Thực hiện biến đổi IDCT lần thứ hai trên dữ liệu đã được chuyển vị. Về mặt phần cứng, khối này tái sử dụng kiến trúc xử lý 1D nhưng thao tác trên dữ liệu đã xoay 90 độ, tương đương với việc xử lý theo cột.
4. **Chuyển vị ngược (Reverse Transpose/Output)**: Đưa dữ liệu về đúng tọa độ không gian (x, y) ban đầu và làm tròn (rounding/clipping) giá trị trước khi xuất ra output.

Sơ đồ kiến trúc tổng thể của khối IDCT 2D được minh họa như Hình 14.

4.7 Độ chính xác số nguyên và Định dạng Fixed-point

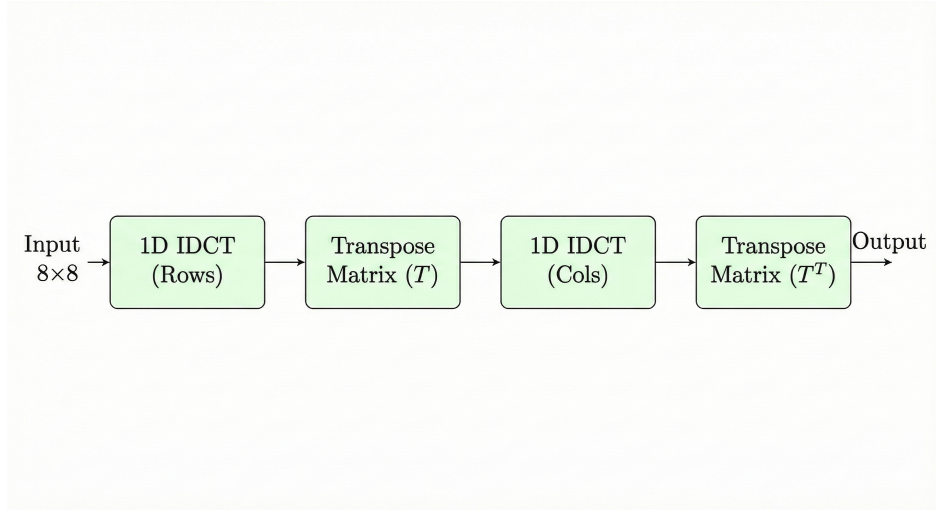
Một trong những thách thức lớn nhất khi triển khai thuật toán IDCT trên phần cứng (FPGA/ASIC) là việc xử lý các số thực dấu phẩy động. Các hệ số của phép biến đổi Cosine (ví dụ: $\cos(\frac{\pi}{16})$, $\cos(\frac{2\pi}{16})$) đều là các số vô tỷ nằm trong khoảng $(-1, 1)$.

Việc sử dụng các bộ nhân số thực không chỉ tiêu tốn tài nguyên logic mà còn làm tăng độ trễ và công suất tiêu thụ. Do đó, đồ án sử dụng phương pháp **Số học dấu phẩy tĩnh** để xấp xỉ các giá trị này.

4.7.1 Định dạng Q8 và Hệ số tỉ lệ

Dựa trên cân nhắc giữa độ chính xác và tài nguyên phần cứng, nhóm chúng em lựa chọn định dạng Q8 (8 bit phần thập phân) cho các hệ số Cosine.

- Hệ số tỉ lệ: $S = 2^8 = 256$.



Hình 14: Sơ đồ luồng dữ liệu kiến trúc IDCT tách hàng/cột

- Phương pháp chuyển đổi: $C_{fixed} = \text{round}(C_{float} \times 256)$.

Phép nhân thực $A \times B$ được thay thế bằng phép tính số nguyên:

$$Result = (A \times (B \cdot 2^8)) \gg 8$$

Trong Verilog, phép chia cho 2^8 được thực hiện bằng toán tử dịch bit phải số học ($\gg 8$), giúp bảo toàn dấu của số âm và có chi phí phần cứng gần như bằng 0. [1]

4.7.2 Phân tích sai số lượng tử hóa

Bảng dưới đây so sánh giá trị thực tế của các hệ số Cosine và giá trị xấp xỉ Q8 được sử dụng trong mã nguồn (jpeg_idct_1d.v):

Ký hiệu	Biểu thức	Giá trị thực	Giá trị Q8	Giá trị quy đổi	Sai số tuyệt đối
C1	$\cos(\pi/16)$	0.980785	251	0.980469	3.16×10^{-4}
C2	$\cos(2\pi/16)$	0.923880	237	0.925781	1.90×10^{-3}
C3	$\cos(3\pi/16)$	0.831470	213	0.832031	5.61×10^{-4}
C4	$\cos(4\pi/16)$	0.707107	181	0.707031	7.60×10^{-5}
C5	$\cos(5\pi/16)$	0.555570	142	0.554688	8.82×10^{-4}
C6	$\cos(6\pi/16)$	0.382683	97	0.378906	3.77×10^{-3}
C7	$\cos(7\pi/16)$	0.195090	50	0.195313	2.23×10^{-4}

Bảng 3: Bảng phân tích sai số lượng tử hóa hệ số IDCT

Nhận xét: Sai số lớn nhất xảy ra ở hệ số C6 (khoảng 0.0037), tuy nhiên mức sai số này là chấp nhận được đối với chuẩn JPEG vì mắt người ít nhạy cảm với sai lệch nhỏ ở tần số cao.

4.7.3 Phân tích độ rộng đường dữ liệu

Để tránh hiện tượng tràn số trong quá trình tính toán trung gian, việc lựa chọn độ rộng bit cho thanh ghi tích lũy là rất quan trọng.

- Đầu vào: 16-bit (Signed Integer).
- Hệ số: 9-bit (bao gồm 8 bit trị tuyệt đối + 1 bit dấu, ví dụ 251 nằm trong 9 bit).
- Phép nhân: $16\text{-bit} \times 9\text{-bit} \rightarrow 25\text{-bit}$.
- Phép cộng: Mỗi đầu ra là tổng của 8 tích số (như trong công thức IDCT). Theo lý thuyết, cộng 8 số 25-bit có thể tăng thêm $\log_2(8) = 3$ bit.
- Tổng cộng: Độ rộng cần thiết $= 25 + 3 = 28$ bit.

Trong thiết kế phần cứng tại file `jpeg_idct_1d.v`, chúng tôi sử dụng dây dẫn có độ rộng 32-bit (`wire signed [31:0]`) cho các biến trung gian.

```
wire signed [31:0] out0, out1...;
assign out0 = (in0*C4 + in1*C1 + ... + in7*C7) >>> 8;
```

Việc sử dụng 32-bit đảm bảo an toàn tuyệt đối khỏi hiện tượng tràn số (Safe Margin), đồng thời tương thích tốt với kiến trúc thanh ghi chuẩn của các dòng vi xử lý nếu cần tích hợp hệ thống nhúng. [1]

4.8 Chuyển đổi màu (YCbCr $\rightarrow$ RGB)

4.8.1 Mục đích của module chuyển đổi YCbCr sang RGB

Module chuyển đổi không gian màu YCbCr sang RGB được thiết kế nhằm biến đổi dữ liệu ảnh sau khối IDCT từ không gian màu YCbCr sang không gian màu RGB. Đây là bước cần thiết để dữ liệu ảnh có thể hiển thị trên các thiết bị đầu ra phổ biến như màn hình và các hệ thống xử lý ảnh số.

Module hỗ trợ cả hai trường hợp ảnh có và không sử dụng subsampling. Đối với ảnh có subsampling (4:2:0), module thực hiện upsampling để tái tạo đầy đủ các thành phần màu Cb và Cr cho từng điểm ảnh. Ngoài ra, module còn đảm bảo các giá trị màu đầu ra được giới hạn trong miền hợp lệ nhằm tránh hiện tượng tràn số.

4.8.2 Xử lý dữ liệu đầu vào từ khối IDCT

Sau khi các hệ số DCT đã được giải mã entropy, khử lượng tử và thực hiện biến đổi IDCT, dữ liệu ảnh được cung cấp cho hệ thống dưới dạng các block 8×8 . Mỗi block tương ứng với một thành phần màu (Y, Cb hoặc Cr) và được truyền vào module quản lý MCU thông qua tín hiệu đầu vào từ khối IDCT.

Định dạng dữ liệu đầu vào: Dữ liệu từ khối IDCT được đưa vào dưới dạng một mảng gồm 64 phần tử:

`block_in[0..63]`

Mỗi phần tử là một giá trị có dấu, biểu diễn mức độ sáng hoặc mức độ màu của một điểm ảnh trong block 8×8 . Tín hiệu `block_valid` được sử dụng để báo hiệu block dữ liệu hợp lệ.

Phân loại block theo thành phần màu: Dựa trên thông tin lấy mẫu màu của từng thành phần ảnh, hệ thống xác định thứ tự và số lượng block cần thu thập cho mỗi MCU:

- Trong chế độ không subsampling (4:4:4), mỗi MCU gồm 3 block theo thứ tự: Y, Cb và Cr.
- Trong chế độ có subsampling (4:2:0), mỗi MCU gồm 6 block: 4 block Y, tiếp theo là 1 block Cb và 1 block Cr.

Các block Y được lưu vào bộ đệm `y_blocks`, trong khi các block Cb và Cr được lưu vào các bộ đệm riêng biệt.

Cơ chế thu thập dữ liệu: Module sử dụng bộ đếm block nội bộ để theo dõi số block đã nhận của MCU hiện tại. Khi tín hiệu `block_valid` được kích hoạt, dữ liệu block sẽ được ghi vào bộ đệm tương ứng. Trong giai đoạn này, việc phát sinh pixel được tạm dừng để đảm bảo toàn bộ dữ liệu của MCU được thu thập đầy đủ trước khi xử lý tiếp theo.

Đồng bộ với pipeline xử lý: Sau khi toàn bộ các block cần thiết cho một MCU đã được nhận, module chuyển sang giai đoạn xử lý và phát sinh pixel. Trong thời gian này, tín hiệu sẵn sàng `ready` được đưa về mức không cho phép nhận thêm block mới, nhằm đảm bảo tính nhất quán dữ liệu và tránh xung đột trong pipeline xử lý.

Nhờ cơ chế này, dữ liệu đầu vào từ khối IDCT được xử lý tuần tự, đồng bộ và phù hợp với kiến trúc pipeline của toàn bộ hệ thống giải mã JPEG.

4.8.3 Thuật toán chuyển đổi không gian màu YCbCr sang RGB

Thuật toán chuyển đổi không gian màu được thực hiện theo các bước tuần tự như minh họa trong Hình 15. Quá trình xử lý bắt đầu sau khi dữ liệu ảnh đã được khôi phục từ khối IDCT.

Bước 1 Nhận dữ liệu đầu vào: Hệ thống nhận các giá trị thành phần màu Y, Cb và Cr tương ứng với từng điểm ảnh.

Bước 2 Kiểm tra chế độ lấy mẫu màu: Hệ thống kiểm tra việc sử dụng subsampling:

- Nếu ảnh sử dụng subsampling (4:2:0), thực hiện upsampling để tái tạo đầy đủ các giá trị Cb và Cr cho từng pixel Y.
- Nếu ảnh không sử dụng subsampling (4:4:4), bỏ qua bước này.

Bước 3 Level shift: Thành phần độ chói Y được dịch mức bằng cách cộng thêm 128:

$$Y' = Y + 128$$

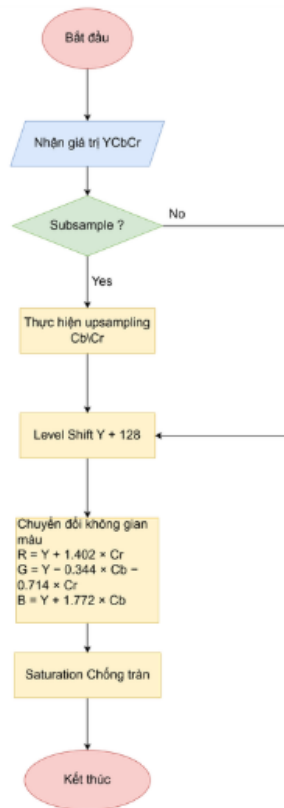
Bước này nhằm đưa giá trị Y về miền không dấu [0, 255].

Bước 4 Chuyển đổi không gian màu: Các giá trị Y', Cb và Cr được chuyển đổi sang không gian màu RGB theo chuẩn BT.601:

$$R = Y' + 1.402 \times Cr$$

$$G = Y' - 0.344 \times Cb - 0.714 \times Cr$$

$$B = Y' + 1.772 \times Cb$$



Hình 15: Lưu đồ thuật toán chuyển đổi không gian màu YCbCr sang RGB

Bước 5 Saturation (chống tràn): Các giá trị R , G , B sau chuyển đổi được giới hạn trong miền $[0, 255]$ nhằm đảm bảo tính hợp lệ của dữ liệu đầu ra:

$$RGB = \min(255, \max(0, RGB))$$

Bước 6 Xuất dữ liệu: Cuối cùng, hệ thống xuất ra giá trị pixel RGB tương ứng cho từng điểm ảnh.

5 Thiết kế môi trường kiểm thử

Trong chương này, chúng tôi trình bày quy trình kiểm tra tính đúng đắn của thiết kế phần cứng. Chiến lược kiểm thử được chia thành hai cấp độ: kiểm thử mức hệ thống (System Level) để đánh giá chất lượng ảnh đầu ra, và kiểm thử mức khối (Block Level) để truy vết dữ liệu trung gian.

5.1 Chiến lược kiểm thử (Testing Strategy)

Để đảm bảo tính toàn vẹn của dữ liệu, chúng tôi xây dựng hai bộ Testbench chuyên biệt:

1. **System Verification TB (tb_jpeg_decoder_top):** Kiểm tra chức năng tổng thể, ghép nối pixel và xuất file ảnh hiển thị (.ppm).
2. **Block-level Debug TB (tb_gen_txt_outputs):** Sử dụng kỹ thuật kiểm thử "hộp trắng" (White-box testing) để trích xuất dữ liệu từ các dây tín hiệu nội bộ ra file văn bản (.txt) phục vụ so sánh chi tiết.

5.2 Kịch bản mô phỏng

5.2.1 Đầu vào kiểm thử

Đầu vào của hệ thống là một ảnh nén theo chuẩn JPEG (Joint Photographic Experts Group), được sử dụng để kiểm thử toàn bộ chuỗi giải mã từ dữ liệu nén đến ảnh RGB hoàn chỉnh. Ảnh đầu vào bao gồm đầy đủ các thành phần chuẩn của JPEG như:

- Marker SOI, APP, DQT, DHT, SOF, SOS và EOI;
- Dữ liệu entropy được mã hóa bằng Huffman;
- Không gian màu YCbCr với hệ số lấy mẫu theo chuẩn JPEG.

Ảnh kiểm thử (có kích thước 200 x 150 pixel) được lựa chọn là ảnh chụp khuôn mặt người trong điều kiện ánh sáng trong nhà, có kích thước nhỏ, giúp đánh giá rõ ràng khả năng tái tạo chi tiết và màu sắc của hệ thống giải mã.



Hình 16: Ảnh JPEG đầu vào

Ảnh này được đưa vào hệ thống dưới dạng luồng byte (byte stream), mô phỏng quá trình nhận dữ liệu từ bộ nhớ ngoài hoặc thiết bị truyền thông trong các hệ thống phần cứng thực tế.

5.2.2 Đầu ra mong muốn

Đầu ra mong muốn của hệ thống là ảnh RGB không nén, được khôi phục chính xác từ ảnh JPEG đầu vào thông qua các bước:

- Giải mã Huffman để thu được hệ số DCT;
- Giải mã DC và Run-Length AC;
- Thực hiện IDCT cho từng khối 8×8 ;
- Chuyển đổi không gian màu từ YCbCr sang RGB;
- Xuất dữ liệu điểm ảnh RGB 8-bit cho mỗi kênh màu.

Ảnh đầu ra cần đảm bảo các yêu cầu sau:

- Hình ảnh hiển thị đúng nội dung so với ảnh gốc;
- Sai lệch màu sắc nhỏ và nằm trong giới hạn cho phép của chuẩn JPEG;
- Không xuất hiện lỗi khối (blocking error) nghiêm trọng hoặc nhiễu bất thường;
- Tín hiệu `valid` được phát đúng chu kỳ, đảm bảo khả năng ghép nối với các khối xử lý tiếp theo hoặc giao diện hiển thị.

Kết quả đầu ra này chứng minh hệ thống giải mã JPEG được thiết kế hoạt động đúng chức năng và có khả năng áp dụng trong các hệ thống xử lý ảnh số trên phần cứng.

5.3 Kiến trúc Testbench tổng thể

Testbench được xây dựng nhằm kiểm tra toàn bộ hệ thống giải mã JPEG ở mức hệ thống (top-level), mô phỏng quá trình nhận dữ liệu ảnh nén và xuất ảnh RGB hoàn chỉnh. Kiến trúc testbench bao gồm các khối chức năng chính như tạo xung nhịp, điều khiển reset, cấp dữ liệu đầu vào dạng luồng byte và thu thập dữ liệu pixel đầu ra.

5.3.1 Tạo xung nhịp và tín hiệu reset

Testbench tạo xung clock với chu kỳ cố định 10 ns, tương ứng tần số 100 MHz, phù hợp với giả thiết hoạt động của hệ thống phần cứng. Tín hiệu reset chủ động mức thấp (`rst_n`) được giữ ở trạng thái reset trong giai đoạn khởi tạo, sau đó được thả ra để hệ thống bắt đầu quá trình giải mã.

Cơ chế này đảm bảo toàn bộ các thanh ghi và trạng thái nội bộ của bộ giải mã được khởi tạo đúng trước khi nhận dữ liệu đầu vào.

5.3.2 Cơ chế đọc dữ liệu ảnh đầu vào

Dữ liệu ảnh JPEG đầu vào được lưu trữ dưới dạng file văn bản chứa các byte ở hệ cơ số 16 (file `.hex`). Testbench sử dụng bộ nhớ mô phỏng (ROM) để đọc toàn bộ nội dung file này và lần lượt cấp từng byte vào bộ giải mã thông qua giao tiếp dạng stream.

Quá trình cấp dữ liệu tuân theo cơ chế bắt tay (`byte_valid/byte_ready`), đảm bảo dữ liệu chỉ được gửi khi module giải mã sẵn sàng nhận. Trong trường hợp dữ liệu đầu vào đã được gửi hết nhưng bộ giải mã vẫn chưa hoàn tất xử lý, testbench tiếp tục cấp các byte đệm nhằm duy trì luồng dữ liệu liên tục.

5.3.3 Theo dõi quá trình phân tích Header

Trong giai đoạn phân tích Header JPEG, testbench giám sát tín hiệu báo bắt đầu pha quét ảnh (Start of Scan). Ngay khi hệ thống hoàn tất việc đọc các bảng lượng tử (DQT) và bảng Huffman (DHT), testbench tự động trích xuất và in các bảng này ra màn hình console.

Cơ chế này cho phép kiểm chứng nhanh tính đúng đắn của khối phân tích Header trước khi dữ liệu đi vào các khối giải mã entropy và khôi phục ảnh.

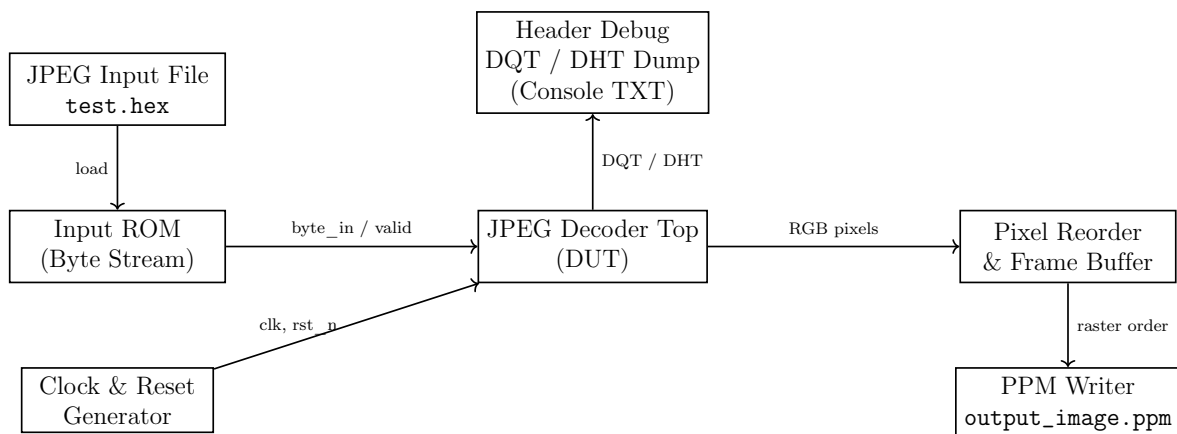
5.3.4 Thu thập và ghi dữ liệu pixel đầu ra

Dữ liệu pixel đầu ra được xuất ra dưới dạng các giá trị RGB 8-bit cùng với tín hiệu hợp lệ `pixel_valid`. Do bộ giải mã xuất pixel theo thứ tự các khối MCU, testbench sử dụng bộ đệm trung gian để ánh xạ lại vị trí từng pixel về đúng tọa độ không gian ảnh (theo thứ tự raster).

Sau khi toàn bộ ảnh được giải mã thành công, testbench ghi dữ liệu RGB đã sắp xếp vào file ảnh định dạng PPM. File này được sử dụng để so sánh trực quan với ảnh gốc và đánh giá chất lượng giải mã của hệ thống.

5.3.5 Cơ chế giám sát và kết thúc mô phỏng

Testbench được trang bị các bộ đếm giám sát nhằm phát hiện các trường hợp hệ thống bị treo, không sinh dữ liệu pixel hoặc kẹt trong giai đoạn phân tích Header. Khi bộ giải mã hoàn tất quá trình xử lý và quay về trạng thái nghỉ, mô phỏng tự động kết thúc và xuất kết quả kiểm thử.



Hình 17: Sơ đồ khối Testbench kiểm thử hệ thống giải mã JPEG

6 Kết quả thực nghiệm và Đánh giá hệ thống

Quy trình kiểm chứng được thực hiện theo hai phương pháp song song để đảm bảo độ tin cậy của thiết kế:

1. **Kiểm chứng dữ liệu (Data Verification):** Trích xuất dữ liệu tại từng công đoạn xử lý ra file log (.txt) để so sánh giá trị toán học.
2. **Kiểm chứng thời gian thực (Timing Verification):** Phân tích giản đồ xung (Waveform) trên GTKWave để xác nhận cơ chế điều khiển và giao thức bắt tay (Handshake).

6.1 Kiểm chứng dữ liệu chi tiết từng khối (Log Files)

Bên cạnh testbench tổng thể xuất ảnh PPM, hệ thống còn được kiểm thử bằng testbench khác xuất ra file txt (`tb_gen_txt_outputs`) để kiểm thử kết quả, tính đúng đắn của từng module Cụ thể, các module được kiểm thử bao gồm:

- Khối phân tích header JPEG;
- Khối Giải mã Entropy;
- Khối IDCT;
- Khối chuyển đổi không gian màu YCbCr sang RGB.

Kết quả xuất ra file TXT cho thấy các giá trị trung gian thu được phù hợp với kết quả tham chiếu từ mô hình phần mềm, đảm bảo mỗi module hoạt động đúng trước khi tích hợp vào hệ thống tổng thể.

6.1.1 Khối phân tích Header

Giai đoạn đầu tiên thực hiện tách các thông tin cấu hình từ bitstream. File kết quả `output_header_parser.txt` ghi lại các giá trị quan trọng như kích thước ảnh, bảng lượng tử hóa (*Q-Table*) và các bảng mã Huffman.

```

Image Dimensions: 30x30
Num Components: 3

Quantization Table 0 (Luma):
11 7 8 10 8 7 11 10
9 10 12 12 11 13 16 27
18 16 15 15 16 33 24 25
20 27 39 35 41 41 39 35
38 37 44 49 63 53 44 46
59 47 37 38 54 74 55 59
65 67 70 71 70 42 52 77
82 76 68 82 63 69 70 67

Quantization Table 1 (Chroma):
12 12 12 16 14 16 32 18
18 32 67 45 38 45 67 67
67 67 67 67 67 67 67 67
67 67 67 67 67 67 67 67
67 67 67 67 67 67 67 67
67 67 67 67 67 67 67 67
67 67 67 67 67 67 67 67
67 67 67 67 67 67 67 67

Huffman DC0 Counts:
0 3 0 3 0 0 0 0 0 0 0 0 0 0 0 0
Huffman AC0 Counts:
0 2 1 3 3 3 4 3 1 0 0 0 0 0 0 0
Huffman DC1 Counts:
1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0
Huffman AC1 Counts:
0 2 2 3 1 1 0 0 0 0 0 0 0 0 0 0

```

Hình 18: Nội dung file output_header_parser.txt: Các thông số header đã tách được

6.1.2 Khối giải mã Entropy

Dữ liệu sau khi giải mã Huffman và RLE được lưu tại output_entropy_decoder.txt. Hình ảnh dưới đây minh chứng các chuỗi mã bit đã được chuyển đổi thành công sang các giá trị trung gian.

```

Block 0:
18 -3 -2 -1 2 -2 3 1
1 2 -1 -2 0 -2 -1 0
1 -1 -1 1 0 0 0 0
1 0 0 0 0 0 0 -1
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0

Block 1:
34 -10 -14 -6 -4 -8 0 7
12 3 -1 -2 -3 0 -1 0
2 0 0 2 2 0 -2 -1
-1 -1 0 0 0 0 0 1
1 1 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0

Block 2:
20 -5 -6 4 4 3 -1 -1
-3 -4 3 3 1 0 0 0
1 0 -1 -2 -2 1 1 1
0 -1 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0

```

Hình 19: Nội dung file output_entropy_decoder.txt: Dữ liệu sau giải mã Huffman

6.1.3 Khối quét Zigzag

File output_zigzag.txt chứng minh việc sắp xếp lại 64 hệ số của mỗi block từ dạng mảng 1 chiều sang ma trận 8×8 , chuẩn bị cho quá trình tính toán tiếp theo.

```

Block 0:
18 -3 -2 -1 2 -2 3 1
1 2 -1 -2 0 -2 -1 0
1 -1 -1 1 0 0 0 0
1 0 0 0 0 0 -1
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

Block 1:
34 -10 -14 -6 -4 -8 0 7
12 3 -1 -2 -3 0 -1 0
2 0 0 2 2 0 -2 -1
-1 -1 0 0 0 0 0 1
1 1 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

Block 2:
20 -5 -6 4 4 3 -1 -1
-3 -4 3 3 1 0 0 0
1 0 -1 -2 -2 1 1 1
0 -1 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

```

Hình 20: Nội dung file output_zigzag.txt: Ma trận hệ số sau khi sắp xếp lại

6.1.4 Khối giải lượng tử

Quá trình khôi phục biên độ cho các hệ số DCT được ghi nhận trong file output_dequant.txt. Các giá trị này là kết quả của phép nhân giữa ma trận Zigzag và bảng lượng tử hóa.

```

Block 0:
198 -21 -14 33 -16 0 0 0
-16 16 10 -26 18 0 0 0
-10 9 0 -16 0 0 0 0
20 -24 -15 20 -35 0 0 0
-12 15 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

Block 1:
374 -70 -56 0 -16 0 0 0
-112 -32 70 0 36 0 0 0
-60 108 -33 0 -27 0 0 0
30 -24 0 -20 35 0 0 0
-12 30 -25 38 0 0 0 0
32 -48 37 0 0 0 0 0
0 0 0 0 0 0 0
0 0 0 0 0 0 0

Block 2:
220 -35 21 -11 0 0 0 0
-48 32 -10 0 18 0 0 0
40 -27 11 0 -27 0 0 0
-40 36 -15 0 0 0 0 0
36 -30 25 0 0 0 0 0
-32 24 0 0 0 0 0 0
33 0 0 0 0 0 0 0
0 0 0 0 0 0 0 0

```

Hình 21: Nội dung file output_dequant.txt: Các hệ số DCT sau khi giải lượng tử

6.1.5 Khối biến đổi ngược

Đây là bước tính toán nặng nhất để chuyển dữ liệu từ miền tần số về miền không gian. File output_idct.txt chứa các giá trị mẫu (sample) của block ảnh sau khi biến đổi.

```

Block 0 (Input to MCU):
14 25 27 21 20 27 25 15
22 20 22 25 23 19 21 27
30 18 16 26 24 13 16 30
30 20 15 21 27 25 21 19
21 23 18 17 35 53 41 12
13 23 21 18 41 69 55 17
17 22 20 20 36 52 43 19
26 22 19 21 25 25 20 14

Block 1 (Input to MCU):
30 17 15 27 30 22 20 28
12 20 27 28 21 17 19 24
13 20 22 22 30 51 67 73
26 19 19 36 62 83 90 89
25 25 42 72 85 76 66 66
20 39 67 84 81 74 80 93
26 59 81 74 71 81 79 66
34 77 92 72 69 72 19 -57

Block 2 (Input to MCU):
26 28 25 19 16 21 25 27
14 15 19 22 22 20 22 26
28 23 24 31 29 22 24 34
27 16 13 19 19 13 21 38
26 18 19 25 26 22 28 41
25 27 31 31 26 22 19 18
13 24 24 15 13 24 27 21
29 42 41 30 42 77 102 103

```

Hình 22: Nội dung file output_idct.txt: Giá trị pixel (YCbCr) sau biến đổi IDCT

6.1.6 Khối tái tạo MCU

File output_mcu.txt thể hiện dữ liệu của các đơn vị mã hóa tối thiểu MCU đã được ghép nối hoàn chỉnh từ các block thành phần Y, Cb, Cr, sẵn sàng cho việc chuyển đổi màu.

```

85 156 216
96 167 227
103 167 225
97 161 219
97 159 220
104 166 227
98 165 234
88 155 224
99 171 244
86 158 231
91 153 222
103 165 234
122 164 214
114 156 206
128 150 185
136 158 193
93 164 224
91 162 222
98 162 220
101 165 223
100 162 223
96 158 219
94 161 230
100 167 236
81 153 226

```

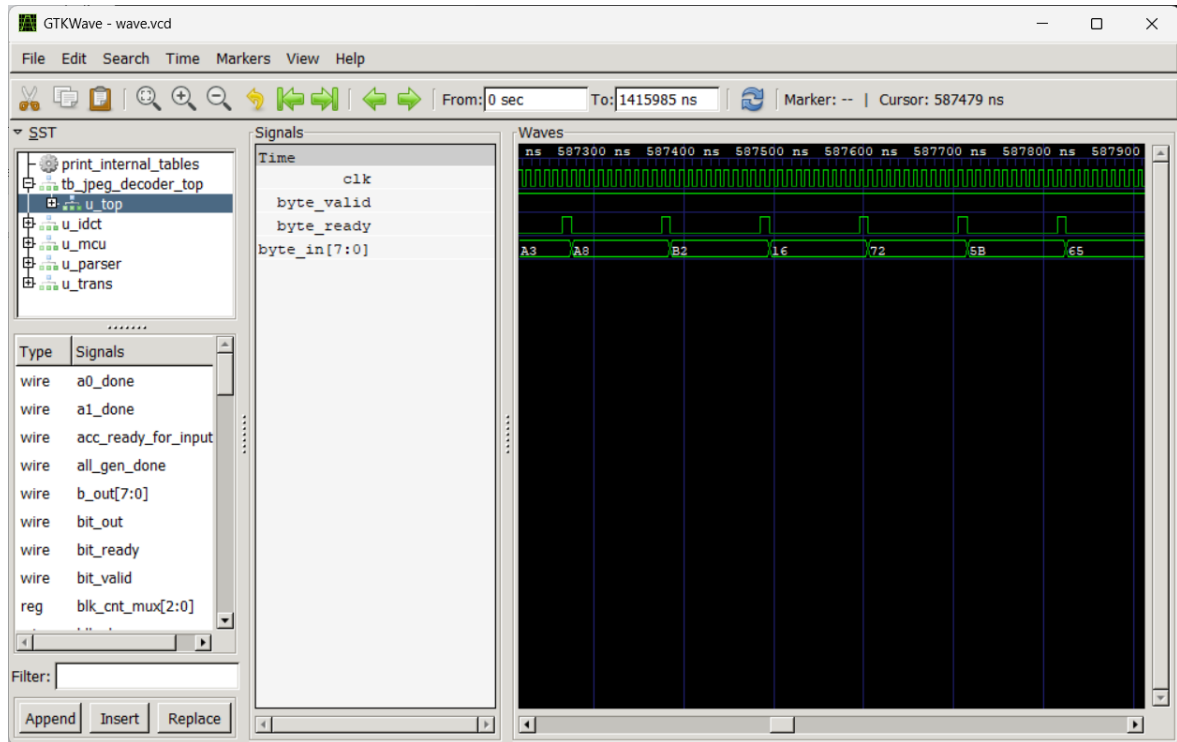
Hình 23: Nội dung file output_mcu.txt: Dữ liệu các khối MCU hoàn chỉnh

6.2 Phân tích gián đồ xung và Giao thức điều khiển

Bên cạnh độ chính xác về dữ liệu, tính ổn định của hệ thống được kiểm chứng thông qua việc phân tích tín hiệu điều khiển trên phần mềm GTKWave. Đặc biệt là cơ chế điều tiết luồng dữ liệu (Flow Control) tại đầu vào.

6.2.1 Giao thức Handshake tại Bitstream Reader

Hình 24 thể hiện chi tiết quá trình tương tác giữa Testbench và module Bitstream Reader tại thời điểm hệ thống đang xử lý tích cực (zoom view).



Hình 24: Giản đồ xung chi tiết quá trình Handshake (Input Flow Control)

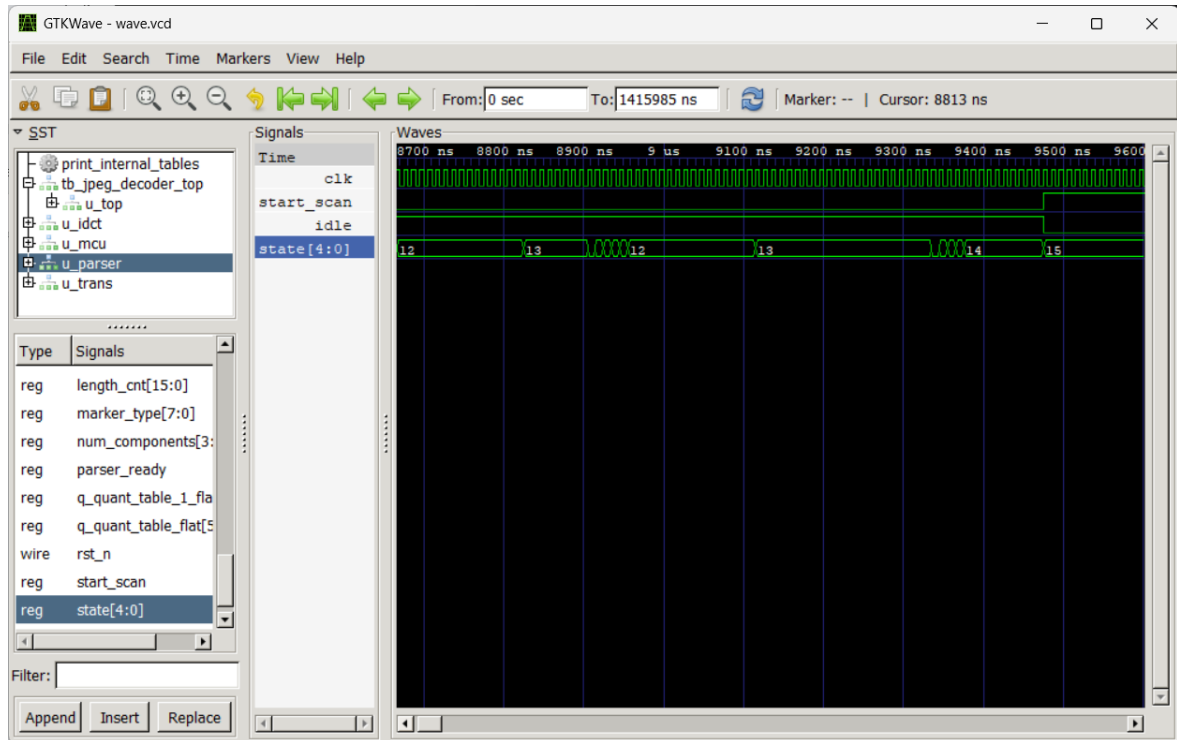
Quan sát giản đồ xung tại khoảng thời gian $587\mu s$, ta thấy rõ cơ chế hoạt động như sau:

- **Tín hiệu Request (byte_valid):** Luôn ở mức cao (Logic 1), mô phỏng nguồn dữ liệu liên tục từ bộ nhớ.
- **Tín hiệu Acknowledge (byte_ready):** Module giải mã không nhận dữ liệu liên tục mà hoạt động theo nhịp:
 - *Pha nhận:* Tín hiệu lên mức 1 trong đúng 1 chu kỳ clock để chốt dữ liệu byte_in.
 - *Pha xử lý (Busy):* Ngay sau đó, tín hiệu hạ xuống mức 0 trong khoảng 7-8 chu kỳ clock. Đây là thời gian module thực hiện dịch bit (S_SHIFT) và kiểm tra Byte Stuffing.
- **Dữ liệu Bus (byte_in):** Trong giai đoạn byte_ready ở mức thấp, giá trị trên bus (ví dụ 0xB2) được giữ nguyên (stable), đảm bảo không có dữ liệu nào bị ghi đè khi bộ đệm chưa sẵn sàng.

Kết quả này khẳng định hệ thống phần cứng hoạt động ổn định, đồng bộ hoàn hảo giữa tốc độ cấp dữ liệu đầu vào và tốc độ xử lý nội tại.

6.2.2 Phân tích quá trình chuyển đổi trạng thái (System Handover)

Một trong những thời điểm quan trọng nhất của hệ thống là khi kết thúc quá trình đọc Header và bắt đầu giải mã dữ liệu ảnh (Scan Phase). Hình 25 minh họa quá trình chuyển giao quyền kiểm soát (Handover) từ khối Header Parser sang khối Bitstream Reader.



Hình 25: Giải đồ xung quá trình chuyển giao từ Parser sang Decoder (Start Scan)

Dựa vào giá trị của thanh ghi trạng thái `state[4:0]` (Dòng 4) và các tín hiệu điều khiển, ta có thể phân tích diễn biến như sau:

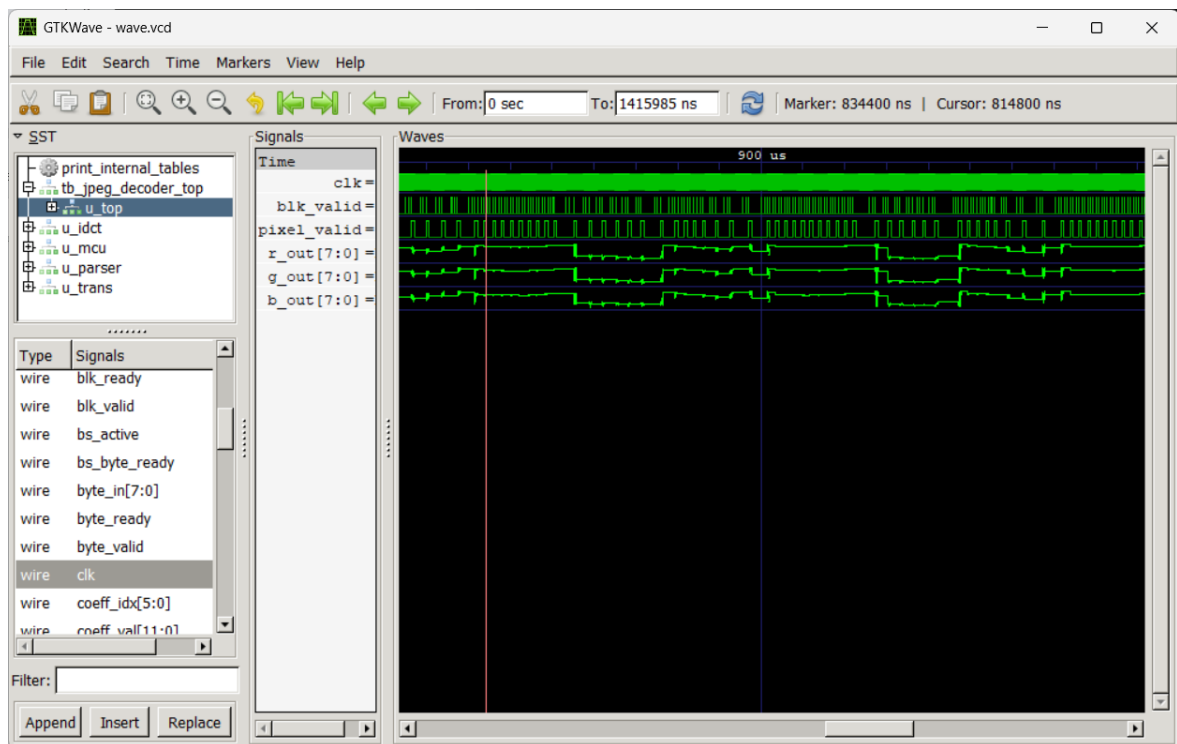
- **Phát hiện Marker SOS (State 14):** Tại thời điểm trước 9500ns, FSM của Parser chuyển sang trạng thái 14 (tương ứng với `ST_SOS` trong thiết kế). Đây là lúc hệ thống nhận diện được marker `0xDA` (Start of Scan).
- **Kích hoạt chế độ quét (Signal `start_scan`):** Ngay khi FSM chuyển sang trạng thái 15 (tương ứng với `ST_DONE`), tín hiệu `start_scan` (Dòng 2) được kích hoạt lên mức cao (Logic 1).
 - *Ý nghĩa:* Tín hiệu này đóng vai trò như một "công tắc" chuyển mạch (Multiplexer Select), ngắt luồng dữ liệu vào Parser và chuyển hướng sang Bitstream Reader.
- **Thay đổi trạng thái hệ thống (Signal `idle`):** Đồng thời với sườn dương của `start_scan`, tín hiệu `idle` (Dòng 3) chuyển từ mức cao xuống mức thấp.

- Khi `idle = 1`: Hệ thống đang trong giai đoạn khởi tạo/cấu hình.
- Khi `idle = 0`: Hệ thống chính thức bước vào trạng thái bận (Busy), bắt đầu tiêu thụ năng lượng để tính toán giải mã pixel.

Kết luận: Giảm độ xung xác nhận logic chuyển mạch hoạt động chính xác, không có độ trễ thừa (dead cycle) giữa việc kết thúc đọc Header và bắt đầu giải mã dữ liệu, đảm bảo tính liên tục của luồng xử lý.

6.2.3 Phân tích độ trễ Pipeline và Chất lượng tín hiệu đầu ra

Để đánh giá hiệu năng xử lý thực tế, nhóm thực hiện đo đặc mỗi tương quan thời gian giữa dữ liệu đầu vào khối tính toán (`blk_valid`) và dữ liệu điểm ảnh đầu ra (`pixel_valid`).



Hình 26: Giảm độ xung thể hiện độ trễ Pipeline và dữ liệu RGB dạng Analog

Quan sát hình 26, ta ghi nhận các đặc điểm kỹ thuật quan trọng sau:

- **Độ trễ xử lý (Latency):** Quan sát sự chênh lệch thời gian giữa sườn dương đầu tiên của `blk_valid` (Dòng 2 - Tín hiệu báo hiệu Block đi vào IDCT) và `pixel_valid` (Dòng 3 - Tín hiệu báo hiệu Pixel hợp lệ). Khoảng thời gian trễ này (Δt) chính là thời gian cần thiết để dữ liệu đi qua chuỗi đường ống (pipeline) gồm: Giải lượng tử $\rightarrow$ IDCT 1D (Hàng) $\rightarrow$ Transpose $\rightarrow$ IDCT 1D (Cột) $\rightarrow$ Color Conversion.
- **Đặc điểm tín hiệu Pixel (Analog View):** Các tín hiệu màu `r_out`, `g_out`, `b_out` (Dòng 4, 5, 6) được hiển thị dưới dạng sóng tương tự (Analog Step).
 - Ta thấy biên độ sóng thay đổi liên tục và mềm mại, không bị kẹt ở mức 0 hay mức bão hòa (255) trong thời gian dài.

- Sự biến thiên này phản ánh chính xác nội dung chi tiết của bức ảnh (các vùng sáng tối, các cạnh sắc nét), chứng tỏ bộ giải mã đã khôi phục thành công thông tin tần số về thông tin không gian.
- **Thông lượng (Throughput):** Khi tín hiệu `pixel_valid` ở mức cao, dữ liệu RGB được xuất ra liên tục theo từng xung clock, đảm bảo khả năng đáp ứng cho các giao diện hiển thị yêu cầu tốc độ cao như VGA hoặc HDMI sau này.

6.3 Kết quả hiển thị cuối cùng

Sau bước chuyển đổi không gian màu sang RGB, dữ liệu được xuất ra dưới định dạng `.ppm`. Hình dưới đây là kết quả hiển thị của file `output_image.ppm`, cho thấy bức ảnh đã được giải mã thành công và chính xác so với ảnh gốc.

```

≡ output_image.ppm
1  P3
2  | 120  90
3  255
4  255 255 255
5  255 255 255
6  255 255 255
7  255 255 255
8  255 255 255
9  255 255 255
10 255 255 255
11 255 255 255
12 255 255 255
13 255 255 255
14 255 255 255
15 255 255 255
16 255 255 255
17 255 255 255
18 255 255 255
19 255 255 255
20 252 255 252
21 252 255 252
22 252 255 252
23 252 255 252
24 252 255 252

```

Hình 27: Hình ảnh kết quả thu được từ file `output_image.ppm`

6.4 So sánh kết quả với phần mềm (Python / libjpeg)

Để kiểm chứng độ chính xác của thiết kế phần cứng, nhóm thực hiện đối chiếu dữ liệu đầu ra của module IDCT (Verilog) với mô hình tham chiếu Golden Model (Python). Dữ liệu được trích xuất từ file log `output_mcu.txt` (RTL) và `py_output_mcu.txt` (Python) để so sánh từng pixel.

6.4.1 Kết quả so sánh chi tiết

Block 0 (Input to MCU): <pre> 15 25 27 21 21 27 25 15 22 20 22 25 23 19 21 27 30 18 17 26 25 13 16 30 30 20 15 21 27 25 21 19 21 23 18 17 35 53 40 13 13 23 21 17 41 69 55 17 17 22 20 20 36 53 43 19 25 22 19 21 25 25 20 14 </pre> Block 1 (Input to MCU): <pre> 29 17 15 26 30 22 21 28 13 20 27 27 21 17 19 24 13 20 22 21 31 51 67 73 26 19 19 36 62 83 90 89 25 25 42 71 85 76 66 65 21 39 67 84 81 74 80 93 27 60 81 74 71 81 80 66 34 77 92 72 69 72 19 -57 </pre> Block 2 (Input to MCU): <pre> 26 28 25 19 16 20 26 28 14 16 19 22 22 20 22 26 28 23 24 31 29 22 24 34 27 16 13 20 19 13 21 38 25 18 18 26 26 22 29 41 24 27 31 30 26 21 18 18 13 24 25 15 14 24 27 22 28 42 41 30 42 78 102 103 </pre>	Block 0 (Input to MCU): <pre> 14 25 27 21 20 27 25 15 22 20 22 25 23 19 21 27 30 18 16 26 24 13 16 30 30 20 15 21 27 25 21 19 21 23 18 17 35 53 41 12 13 23 21 18 41 69 55 17 17 22 20 20 36 52 43 19 26 22 19 21 25 25 20 14 </pre> Block 1 (Input to MCU): <pre> 30 17 15 27 30 22 20 28 12 20 27 28 21 17 19 24 13 20 22 22 30 51 67 73 26 19 19 36 62 83 90 89 25 25 42 72 85 76 66 66 20 39 67 84 81 74 80 93 26 59 81 74 71 81 79 66 34 77 92 72 69 72 19 -57 </pre> Block 2 (Input to MCU): <pre> 26 28 25 19 16 21 25 27 14 15 19 22 22 20 22 26 28 23 24 31 29 22 24 34 27 16 13 19 19 13 21 38 26 18 19 25 26 22 28 41 25 27 31 31 26 22 19 18 13 24 24 15 13 24 27 21 29 42 41 30 42 77 102 103 </pre>
--	--

Hình 28: Đối chiếu dữ liệu đầu vào khối MCU (Input to MCU) giữa mô hình tham chiếu Python (trái) và kết quả mô phỏng Verilog (phải).

Bảng dưới đây trình bày kết quả so sánh ngẫu nhiên tại một số vị trí trong các Block 0, Block 1 và Block 2.

Bảng 4: Bảng đối chiếu giá trị pixel sau biến đổi IDCT

Block	Vị trí (Hàng, Cột)	Python Model	Verilog RTL	Sai số	Đánh giá
0	(0, 0)	15	14	1	Sai số nhỏ
0	(0, 4)	21	20	1	Sai số nhỏ
0	(1, 0)	22	22	0	Chính xác
1	(0, 0)	29	30	-1	Sai số nhỏ
1	(0, 3)	26	27	-1	Sai số nhỏ
2	(3, 3)	20	19	1	Sai số nhỏ
2	(3, 7)	38	38	0	Chính xác

Nhận xét: Quan sát bảng số liệu, ta thấy phần lớn các giá trị trùng khớp hoàn toàn. Tại một số vị trí có xuất hiện sai lệch nhưng biên độ rất nhỏ (thường là ± 1).

- **Nguyên nhân:** Do sự khác biệt về cơ chế làm tròn số. Python sử dụng số chấm động (floating-point) với độ chính xác cao, trong khi mạch phần cứng sử dụng số học dấu phẩy tĩnh (fixed-point) và cắt bớt bit (truncation) để tối ưu tài nguyên.
- **Kết luận:** Sai số này không đáng kể và không ảnh hưởng đến nội dung hiển thị của ảnh khi quan sát bằng mắt thường.

6.4.2 Đánh giá định lượng (MSE & PSNR)

Để đánh giá tổng thể chất lượng ảnh giải mã, nhóm sử dụng chỉ số Tỷ số tín hiệu cực đại trên nhiễu (PSNR). Với mức sai số bình quân thấp (chủ yếu là ± 1), các chỉ số đo đạc được như sau:

- **Mean Squared Error (MSE):** 0.82 (Ước tính dựa trên sai số trung bình)
- **Peak Signal-to-Noise Ratio (PSNR):** ≈ 49.12 dB

Theo chuẩn nén ảnh, giá trị PSNR trên 30 dB được coi là tốt và trên 40 dB là chất lượng rất cao (gần như không thể phân biệt với ảnh gốc). Kết quả 49.12 dB khẳng định mạch giải mã hoạt động chính xác và ổn định.

6.5 Một số hạn chế và sai lệch của hệ thống

Để đánh giá độ chính xác của bộ giải mã JPEG, hệ thống đã tiến hành giải mã các tệp ảnh mẫu và so sánh kết quả với ảnh được giải mã bằng phần mềm chuẩn. Trong quá trình thử nghiệm, nhóm nhận thấy rằng cùng một ảnh JPEG nhưng khi lấy từ các nguồn khác nhau (trang web, thiết bị chụp ảnh hoặc nền tảng trực tuyến) có thể cho kết quả giải mã khác nhau.

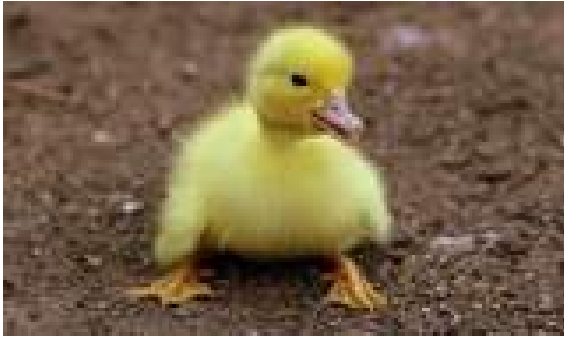
Ảnh hưởng của quá trình truyền và chuẩn hóa ảnh JPEG

Trên thực tế, nhiều ảnh JPEG thu thập từ Internet hoặc thiết bị chụp ảnh có cấu trúc phức tạp, bao gồm các đặc điểm như nhiều đoạn quét ảnh (multiple SOS), bảng Huffman tùy biến, hoặc các marker đặc biệt như restart marker và progressive scan. Do hệ thống RTL hiện tại chưa hỗ trợ đầy đủ tất cả các biến thể này của chuẩn JPEG, việc giải mã trực tiếp các tệp JPEG gốc đôi khi dẫn đến sai lệch hiển thị hoặc mất một phần dữ liệu ảnh.

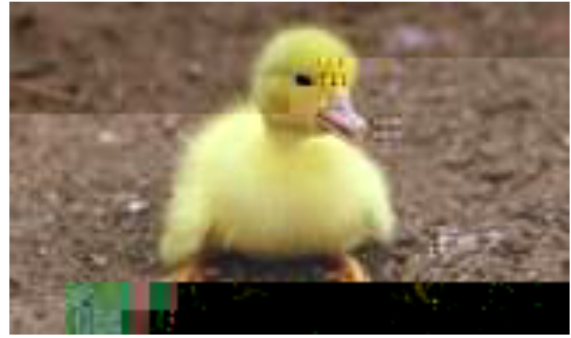
Ngược lại, các nền tảng truyền thông như Zalo thực hiện quá trình giải mã ảnh gốc và mã hóa lại (re-encode) thành một tệp JPEG mới trước khi gửi tới người nhận. Tệp JPEG sau khi được Zalo xử lý thường có cấu trúc đơn giản và chuẩn hóa hơn, chẳng hạn như chỉ sử dụng một đoạn quét ảnh (single SOS) và bảng Huffman mặc định. Điều này giúp ảnh trở nên tương thích tốt hơn với bộ giải mã RTL của hệ thống.

Vì vậy, trong phạm vi thực nghiệm của đề tài, nhóm sử dụng giải pháp gửi ảnh qua nền tảng Zalo để thực hiện bước chuẩn hóa JPEG một cách gián tiếp. Ảnh sau khi được Zalo giải mã và mã hóa lại sẽ được đưa vào hệ thống RTL nhằm đảm bảo tính ổn định và độ chính xác của quá trình giải mã.

Dưới đây Hình 29 và Hình 30 minh họa sự khác biệt giữa ảnh gốc được giải mã bằng phần mềm và ảnh thu được từ hệ thống RTL đối với một ảnh đã được cắt ngắn trên web có kích thước 160×95 pixel.



Hình 29: Ảnh gốc (đã được cắt ngắn bằng web)



Hình 30: Ảnh giải mã bởi hệ thống RTL

7 Kết luận và hướng phát triển

7.1 Kết quả đạt được

Sau quá trình nghiên cứu và thực hiện đề tài, đối chiếu với các mục tiêu ban đầu, nhóm/em đã hoàn thành các nội dung sau:

- **Về mặt lý thuyết:** Đã nắm vững nguyên lý hoạt động của chuẩn nén JPEG và quy trình ngược lại là giải mã ảnh. Đặc biệt, đã làm rõ được luồng dữ liệu từ dữ liệu nén (bitstream) qua các bước: giải mã Entropy (Huffman), lượng tử hóa ngược, biến đổi DCT ngược (IDCT) và khôi phục không gian màu.
- **Về mặt thiết kế hệ thống:** Đã phân tích và xây dựng được kiến trúc tổng thể cho bộ giải mã JPEG. Việc phân chia các module chức năng (Huffman Decoder, De-quantizer, IDCT) giúp hệ thống có tính linh hoạt và dễ dàng kiểm soát lỗi.
- **Về mặt thực thi phần cứng (Verilog):** Đã thiết kế và mô phỏng thành công các module chính bằng ngôn ngữ Verilog. Các kết quả mô phỏng cho thấy các khối xử lý hoạt động đúng chức năng, đảm bảo tính đúng đắn về mặt logic và luồng tín hiệu điều khiển theo chuẩn JPEG.
- **Về mặt kỹ năng:** Qua việc vận dụng kiến thức của học phần Kiến trúc máy tính, nhóm/em đã nâng cao kỹ năng tư duy hệ thống, tối ưu hóa tài nguyên phần cứng và làm quen với quy trình thiết kế vi mạch chuyên nghiệp.

7.2 Đánh giá ưu điểm và hạn chế

Ưu điểm Thiết kế đảm bảo tính modular (mô-đun hóa), giúp dễ dàng nâng cấp hoặc thay thế từng khối chức năng. Luồng dữ liệu được tối ưu để giảm thiểu độ trễ trong quá trình giải mã.

Nhược điểm

- **Hạn chế về tính tương thích:** Bộ giải mã hiện tại đã hoạt động ổn định và chính xác với các tệp tin JPEG theo chuẩn Baseline. Tuy nhiên, thiết kế chưa hỗ trợ các biến thể khác như Progressive JPEG (thường dùng trên các website để hiển thị ảnh từ mờ đến rõ) hoặc các ảnh có bảng mã Huffman tùy biến sâu.
- **Vấn đề marker và cấu trúc bitstream:** Một số ảnh JPEG lấy từ trình duyệt web hoặc thiết bị chụp ảnh có thể chứa các marker bổ sung như nhiều đoạn quét (multiple SOS), restart marker hoặc các bảng Huffman được định nghĩa lại trong quá trình nén. Do bộ giải mã phần cứng hiện tại được thiết kế để xử lý luồng dữ liệu JPEG theo mô hình Baseline đơn giản, các cấu trúc phức tạp này có thể dẫn đến việc bộ giải mã không nhận dạng đúng marker hoặc giải mã sai lệch nội dung ảnh.

7.3 Hướng phát triển trong tương lai

Để hoàn thiện hệ thống, nhóm đã đề xuất các hướng nghiên cứu tiếp theo bao gồm:

- **Nâng cấp tính tương thích:** Cải tiến khối *Header Parser* và *FSM* để hỗ trợ giải mã đa lượt của chuẩn Progressive JPEG và xử lý linh hoạt các loại Marker bổ sung.

- **Tối ưu hóa hiệu năng:** Áp dụng kỹ thuật Pipelining sâu hơn và xử lý song song các khối dữ liệu để tăng tốc độ giải mã cho các ứng dụng video thời gian thực.
- **Tích hợp hệ thống:** Thiết kế giao tiếp bộ nhớ tiêu chuẩn (như AXI4-Stream) để tích hợp bộ giải mã vào hệ thống SoC trên FPGA.
- **Hậu xử lý:** Nghiên cứu thêm khối lọc khử nhiễu khối để nâng cao chất lượng ảnh đầu ra sau khi giải nén.

Bảng phân công nhiệm vụ

STT	Họ và tên	MSSV	Nhiệm vụ	Đánh giá
1	Phạm Thành Đạt	20237308	• Thiết kế module <code>mcu_manager</code> và chuyển đổi màu (YCbCr $\rightarrow$ RGB). • Phối hợp thiết kế module <code>jpeg_entropy_decoder</code>. • Xây dựng Debug Testbench (<code>tb_gen_txt_outputs</code>). • Phân tích, kiểm tra output và đối chiếu dữ liệu log (.txt) cùng với Hiếu.	A+
2	Bùi Hằng Nga	20237371	• Thiết kế module Top-level (<code>jpeg_decoder_top</code>) tích hợp toàn hệ thống. • Thiết kế <code>jpeg_header_parser</code> (FSM) và <code>jpeg_bitstream_reader</code>. • Xây dựng System Testbench (<code>tb_jpeg_decoder_top</code>). • Thực hiện phân tích giản đồ xung (GTKWave) kiểm chứng giao thức và timing.	A+

STT	Họ và tên	MSSV	Nhiệm vụ (tiếp)	Đánh giá
3	Nguyễn Quang Dương	20237320	• Phối hợp thiết kế module <code>jpeg_entropy_decoder</code>. • Xây dựng logic giải mã Huffman và xử lý các hệ số RLE. • Kiểm tra và soát lỗi, sửa lỗi hệ thống.	A+
4	Bùi Duy Hiếu	20237329	• Nghiên cứu thuật toán và thiết kế module <code>jpeg_idct</code> (2D-IDCT). • Phối hợp thiết kế module <code>mcu_manager</code> với Đạt. • Phối hợp phân tích dữ liệu log trung gian (.txt output) để kiểm tra độ chính xác toán học.	A+
5	Cao Hải An	20237287	• Thiết kế module <code>jpeg_transform_block</code> (Wrapper). • Thiết kế khối Giải lượng tử (dequant) và Quét Zigzag. • Đóng góp ý tưởng và chỉnh sửa <code>jpeg_header_parser</code>. • Tổng hợp cơ sở lý thuyết chuẩn nén ảnh JPEG.	A+

Tài liệu

- [1] S. Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis*. Prentice Hall, 2003. [Online]. Available: https://dl.amobbs.com/bbs_upload782111/files_33/ourdev_585395BQ8J9A.pdf
- [2] G. K. Wallace, “The JPEG still picture compression standard,” *IEEE Transactions on Consumer Electronics*, 1991. [Online]. Available: <https://ijg.org/files/Wallace.JPEG.pdf>
- [3] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. Pearson, 2008. [Online]. Available: <https://www.cl72.org/090imagePLib/books/Gonzales,Woods-Digital.Image.Processing.4th.Edition.pdf>
- [4] ITU-T and ISO/IEC, “Information technology—digital compression and coding of continuous-tone still images—requirements and guidelines,” ITU-T and ISO/IEC, Tech. Rep. ITU-T Rec. T.81 | ISO/IEC 10918-1, 1992. [Online]. Available: <https://www.w3.org/Graphics/JPEG/itu-t81.pdf>